ENGINEERING SPECIFICATION

Fine-Grained Access Platform

Decision, enforcement, documents
and key custody for agents, tools and storage

Version	2.2 (draft for review), 30 October 2026. Supersedes 2.1, 2.0 and 1.3. Records decisions D-01 to D-03: our own policy engine and format, our own object storage, and upstream projects used for ideas only.
Status	Restructured specification. No implementation exists. The test vectors have not been run against code, and policy-evaluation vectors marked for oracle confirmation are not yet frozen.
Structure	Seven parts (0–6) built in three releases: R1 (decision core, enforcement, content and MCP over our own single-node store), R2 (key custody), R3 (S3 and replicated storage). Each requirement has one obligation, a phase and a verifying test.
Core language	Go; one module; roles built from role-specific entrypoints.
Open decisions	Phase 0 decisions D-04 to D-11 (Section 0.6) must be closed before the work they gate; D-01 to D-03 are decided.
Deliverables	This LaTeX source and PDF; `vectors/` (machine-readable test vectors); `tools/check_spec.py` (consistency check); generated tables.

Central principle. Every protected operation is an explicit action on a resource, evaluated with trusted identity, delegation, resource state and execution context. An enforcement point binds the resulting decision to the actual operation. Encryption and key release use the same model, with limits on revocation and offline use stated rather than implied.

The component prefix `fga` is a placeholder, not a product or trademark selection. All described modules and APIs are proposed project code unless identified as external dependencies. Items carried over from version 1.3 that this revision could not re-verify (version pins, commit hashes, URLs, upstream license terms) are listed in Appendix F.

Contents

0 Overview, guarantees and threat model **1**
0.1 Product statement and scope boundary . . . 1
0.2 Releases and exit gates . . . 1
0.3 Guarantees and non-guarantees . . . 2
0.4 Threat model . . . 3
0.5 Architecture and roles . . . 6
0.6 Phase 0 decisions . . . 6
0.7 Conventions . . . 7

1 Decision core **8**
1.1 Identity and trusted context . . . 8
1.2 Delegation . . . 8
1.3 Resources, actions and canonical identity . . . 9
1.4 Policy evaluation . . . 10
1.4.1 Evaluator contract . . . 10
1.4.2 Policy format . . . 12
1.5 Decision API . . . 13
1.6 Policy releases and readiness . . . 14
1.7 State and catalog . . . 15
1.8 Audit . . . 16
1.9 Administration . . . 17
1.10 Availability of authority state . . . 17

2 Enforcement points **18**
2.1 Adapter contract . . . 18
2.2 Trust propagation . . . 19
2.3 Tool executor . . . 20
2.4 Approvals and reservations . . . 21
2.5 Invocation state machine and unknown outcomes . . . 22
2.6 Data flow and the context high-water mark . . . 22
2.7 MCP transport and surface . . . 23
2.8 HTTP, service and future adapters . . . 24

3 Content, documents and disclosure **24**
3.1 Content service . . . 24
3.2 Publication protocol . . . 25
3.3 Object backend contract . . . 26
3.4 Documents and text units . . . 27
3.5 Labels . . . 27
3.6 Disclosure . . . 28
3.7 Search . . . 29
3.8 Derived outputs and declassification . . . 29
3.9 Sizing and scaling . . . 30

4 Key custody **31**
4.1 Two custody profiles . . . 31
4.2 Cipher suite and immutable binding . . . 32
4.3 Key hierarchy and broker . . . 32
4.4 KMS provider contract . . . 34
4.5 Streaming, retries and copy . . . 34
4.6 Rotation, compromise, revocation and recovery . . . 35
4.7 Remote recipients and HPKE delivery . . . 35

5 S3 compatibility and replicated storage **37**
5.1 S3 compatibility profile v1 . . . 37
5.2 Replicated blob backend . . . 38

6 Engineering and verification **39**
6.1 Source provenance and licensing . . . 40
6.2 Code quality and naming . . . 40
6.3 Libraries and dependencies . . . 41
6.4 Module and role structure . . . 42
6.5 Keeping the code base small . . . 43
6.6 Testing . . . 43
6.6.1 Security and correctness matrix . . . 44
6.7 Performance qualification . . . 46
6.8 Release gates . . . 46

A Representative end-to-end scenarios **47**
A.1 An agent reads a document and submits an expense (R1-B, R1-C) . . . 47
A.2 Revocation during an upload (R1-C) . . . 47
A.3 A remote AI reads only approved sections (R1-C; export in R2) . . . 48
A.4 A retrieved document tries to give instructions (R1-C) . . . 48
A.5 The context high-water mark (R1-C) . . . 48
A.6 An external outcome is unknown (R1-B) . . . 48
A.7 Storage-node compromise (R2, R3) . . . 48
A.8 One failure domain is lost (R1-C; blobs in R3) . . . 49

B Test vectors **49**

C Traceability **57**

D Review response (2.0 to 2.1) **59**

E Mapping from version 1.3 **61**

F Carried-over items not verified in this revision **65**

0 Overview, guarantees and threat model

0.1 Product statement and scope boundary

The platform decides and enforces who may do what to which resource. It serves people, agents acting for people, tools, MCP clients and HTTP clients, and in a later release S3 clients. Every protected operation is an explicit action on a resource, evaluated with trusted identity, delegation, resource state and execution context. An enforcement point binds the result to the actual operation. Document text, search, model submission and key release use the same decision model.

"Access for everything" means a common resource/action vocabulary and a small adapter contract. It does not mean that one server controls arbitrary operating-system calls, third-party credentials, databases or files outside its enforcement paths. A resource is protected only when every relevant access route is mediated or independently restricted.

[OVR-001][NEW] Every release publishes an enforcement coverage matrix that lists each access route to protected resources (adapter, operation, resource type, action, mediation point, verifying test). A route that is absent from the matrix **MUST NOT** be described as protected. (P0 · REV)

[OVR-002][NEW] Each threat in Table 4 maps to at least one requirement and one test, and each guarantee in Table 2 maps to at least one requirement and one test. The specification check fails when a mapping is missing or names an undefined identifier. (P0 · LINT)

[OVR-003][NEW] Product documentation and external claims **MUST NOT** state more than the guarantees in Table 2 and **MUST** repeat the non-guarantees in Table 3 wherever revocation, model-output control, encryption or storage durability is described. (P0 · REV)

Out of scope for this specification: attribute-based and proxy re-encryption, erasure-coded storage, compiling policy into database queries, general file-content redaction, FTP/SFTP, S3 Select, tiering, site replication, and mediation of unrestricted local shells or direct local file reads.

0.2 Releases and exit gates

The original design bundled five products. This revision orders them by risk and dependency. A release is accepted only when its exit tests (Section 6.6.1) pass; later releases do not weaken earlier gates.

Phase	Delivers	Exit tests
R1-A	**Decision core.** Identity and delegation, resource registry, policy evaluation in the production profile, Decide/DecideBatch with signed decisions, policy releases, catalog, audit outbox. Part 1.	T-01, T-02, T-03, T-13 (admission cases), T-14, T-15 (audit cases), T-19, T-20
R1-B	**Enforcement.** Tool executor, registry, approvals, reservations, invocation state machine, one real connector. Part 2.	T-04, T-05
R1-C	**Content, documents and MCP over our own single-node store.** Content service, publication protocol, labels, units, disclosure, search, derived outputs, context high-water mark, control-plane availability. Parts 2 (MCP) and 3.	T-06, T-08, T-09, T-13, T-15, T-16, T-18, T-21, T-22

Phase	Delivers	Exit tests
R2	**Key custody.** Envelope encryption, key broker, KMS provider, recipient registry, HPKE export. Part 4.	T-10, T-11, T-12, T-13, T-22
R3	**S3 and replicated storage.** SigV4 profile, S3 handlers, replicated blob backend, repair, evacuation. Part 5.	T-07, T-08, T-15, T-17, T-18

Phase P0 labels process requirements that apply from the first commit. In R1, at-rest protection of blobs comes from the host's disk encryption, and the platform does not advertise per-unit cryptographic separation; that claim starts with R2. Everything runs locally for now; the multi-domain topology is a later target.

R1-A, R1-B and R1-C are subphases of one release: none is deployed to production on its own (GATE-003). Where a test is shared across subphases, its cases are assigned to the earliest subphase whose requirements claim it as their verification (Table 20), and the specification check rejects a requirement whose test first applies in a later phase.

0.3 Guarantees and non-guarantees

Table 2 is the complete list of what the platform promises. Each row is conditional on its requirements being implemented and its tests passing. Table 3 lists limits that recur throughout the document; they are stated here once.

Table 2: Guarantees

ID	Guarantee	Requirements	Tests
G-01	Fail closed. A protected operation runs only on an explicit, complete `ALLOW` bound to the exact operation.	DEC-002, ENF-001, ENF-003	T-01, T-04
G-02	One decision, many routes. The same normalized request gets the same business decision through every adapter.	RES-002, RES-003, CNT-001, MCP-002	T-01, T-06
G-03	Fresh authority. A stale policy bundle, stale identity or delegation revision, or expired decision never authorizes; authority is rechecked at admission and at the irreversible checkpoint.	DEC-005, DEC-006, REL-003	T-13, T-14
G-04	Platform ceilings. No principal, role or tenant-scope policy result can override a platform ceiling or tenant isolation.	POL-007, DAT-001	T-01, T-02
G-05	Bound approvals and budgets. Approvals and reservations are single-use, bound to the exact operation, and consumed atomically.	APR-001, APR-002, APR-004	T-04, T-05
G-06	No assumed outcomes. An external effect with an unknown result is held as `UNKNOWN` and resolved by a recorded action; it is never treated as success or failure.	INV-003, INV-006, INV-007	T-05

ID	Guarantee	Requirements	Tests
G-07	Unit-level disclosure. A denied unit does not reach a recipient through any supported disclosure route (search, read, citation, model context, derived output).	DSC-001, DSC-002, DRV-002	T-09
G-08	Durable evidence. Decisions, key releases, mutations and approval consumption leave audit records that do not silently drop.	AUD-001, AUD-002	T-15

Table 3: Non-guarantees

ID	Limit	Requirements
N-01	Revocation does not recall bytes already emitted, exported keys, retained plaintext or context already received by a model provider. Exposure on a live stream is bounded by the stream lease.	KEY-010, CNT-008
N-02	No guarantee about what a model infers, says or sends. Cross-tool flow is constrained only for mediated tools, through the execution-context high-water mark, and only for runtimes that preserve their context lifetime; unmediated egress and content that reaches the agent by other routes are not covered.	FLW-001, FLW-003, FLW-005, FLW-008
N-03	Server-managed encryption does not exclude storage-service operators from plaintext. It protects against reading storage disks and backups.	CRY-001
N-04	No defence against a host operator who can replace the running software, or against a holder of valid KMS credentials beyond key-release audit.	ADM-003, KEY-007
N-05	No exactly-once guarantee across arbitrary external APIs.	INV-005
N-06	No claim that a policy expresses its owner's intent. The platform enforces what the policy says and reports effect changes on the conformance corpus.	REL-005
N-07	No regulatory WORM or Object Lock claim, no universal document parsing, and no claim that default S3 client settings work.	PUB-009, DOC-004, S3P-001
N-08	R1 stores blobs on one node. Losing that node loses blobs written since the last backup, so R1 makes no zero-loss claim for blob content.	BKD-002, AVL-005

0.4 Threat model

Assets. A1 protected content and plaintext; A2 keys and key material; A3 policy bundles, identity, delegation and other authority state; A4 approvals, budgets and invocation state; A5 audit evidence; A6 credentials for downstream tools.

Adversaries. E1 malicious or curious tenant user; E2 manipulated agent or model (prompt injection, hostile retrieved content); E3 compromised or malicious tool server, connector or downstream service; E4 compromised gateway or content-service process; E5 reader of storage disks or backups; E6 holder of broker or KMS credentials; E7 over-privileged or malicious administrator, including a policy author; E8 network attacker; E9 recipient that retains exported keys or plaintext. A host operator able to replace the running software (E10) is out of scope (ADM-003).

Trust boundaries. TB1 client and adapter; TB2 adapter and decision service; TB3 gateway or content service and key broker; TB4 executor, connector and downstream service; TB5 storage processes and blob backend; TB6 control plane and administrators. Section 2.2 lists the evidence that crosses each boundary.

Table 4: Threats

ID	Threat	Who	Mitigations	Tests
TH-01	Forged identity, role, tenant or delegation supplied as authority.	E1, E2	ID-001, ID-002, DEC-007	T-03
TH-02	Policy fails open: an erroring or skipped deny rule, or a missing attribute, lets another rule allow.	E7	POL-005, POL-006	T-02
TH-03	A principal-policy, role or tenant-scope result overrides a platform ceiling.	E7	POL-007	T-01, T-02
TH-04	Stale policy or identity state authorizes after revocation or policy change.	E1	REL-003, DEC-006	T-13, T-14
TH-05	Arguments, target or amount change between approval and dispatch.	E2, E3	AGT-004, APR-001	T-04
TH-06	The approver is shown a description that differs from what executes.	E2	APR-005	T-04
TH-07	Double spend or duplicate dispatch under concurrency or retry.	E1, E2	APR-004, INV-002	T-05
TH-08	An unknown external outcome is treated as failure and retried, or leaks a reservation.	E3	INV-003, INV-006, INV-007	T-05
TH-09	Retrieved content instructs the agent to change the registry, widen delegation or call another tool.	E2	MCP-004, AGT-001	T-03, T-04
TH-10	A denied unit leaks through search, citations, counts, ranks or derived output.	E1, E2	DSC-001, SRH-004, DRV-002	T-09
TH-11	Data read through one tool is sent out through another.	E2	FLW-002, FLW-003, FLW-007	T-21
TH-12	A compromised gateway asks the broker for keys it should not receive.	E4	DEC-008, TRU-003, KEY-002	T-03, T-11

ID	Threat	Who	Mitigations	Tests
TH-13	A storage disk or backup is read.	E5	CRY-002, CRY-003	T-10, T-18
TH-14	A recipient keeps keys or plaintext after revocation.	E9	KEY-010, CNT-008	T-13
TH-15	An orphan or unpublished blob becomes readable, or a publication race publishes obsolete state.	E1, E4	PUB-003, PUB-004	T-08
TH-16	Existence or metadata of a hidden object is inferred from errors, HEAD or listings.	E1	CNT-004, CNT-005	T-09
TH-17	Secrets or protected content appear in logs, traces, dumps or support bundles.	E6, E7	KEY-007, AUD-003	T-19
TH-18	A less-trusted role gains privileges of a more-trusted role through the shared code base.	E4	MOD-002, MOD-004	T-20
TH-19	An ordinary API request disables the authorizer or grants itself administrator rights.	E7	ADM-002	T-20
TH-20	A signed decision or continuation token is replayed or used by another party.	E4, E8	DEC-008, CNT-005	T-03, T-09
TH-21	A stale standby or split brain restores revoked grants or loses acknowledged writes.	E8	AVL-002, AVL-003	T-16
TH-22	Mandatory audit records are lost under load or exporter failure.	E7	AUD-002	T-15
TH-23	A connector follows a redirect or rebinding to a destination that was not approved.	E3	AGT-005	T-04
TH-24	An HPKE recipient key is substituted or an unregistered key is activated.	E1, E6	RCP-001, RCP-002, RCP-003	T-03, T-11
TH-25	A derived output re-enters flow control with a label lower than its sources.	E2	DRV-007, DRV-008, FLW-003	T-21
TH-26	Egress is dispatched on a stale context mark while a concurrent read raises it.	E2	FLW-007	T-21
TH-27	A transaction left visible but unreplicated by a cancelled synchronous wait is treated as durable.	E8	AVL-004, INV-002	T-16

0.5 Architecture and roles

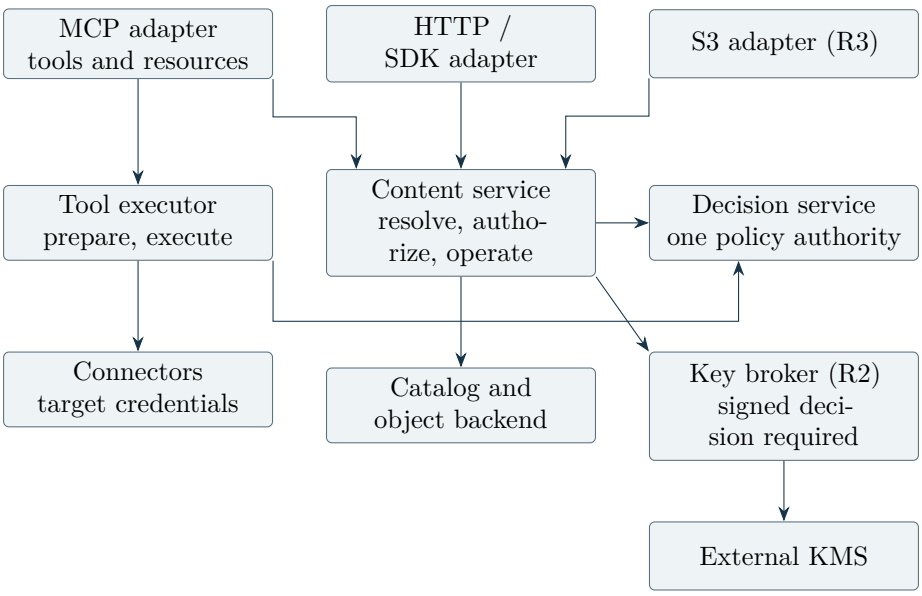


Figure 1: One access-controlled system, several protocol interfaces. Process isolation and replication do not create different resources or permission systems.

Role (`fga serve -role=`)	Hosts	Phase
`decision`	Decide/DecideBatch, context resolver, snapshot activation, decision-signing key	R1-A
`control`	Control API, catalog administration, approvals, reservations, policy releases	R1-A
`tools`	Tool executor, connectors, MCP adapter	R1-B
`storage`	Content service, HTTP handlers, external-backend adapter (S3 handlers in R3)	R1-C
`keys`	Key broker, KMS client, recipient registry	R2
`blob`	Replicated blob service and repair (when the custom backend is selected)	R3

Roles are deployment and security boundaries of one product, not separate permission domains. The decision service is the only evaluator; no adapter, executor or broker evaluates policy.

0.6 Phase 0 decisions

These decisions are open. Each has a default so that work can be planned, but the default is not a decision until the owner records it. Work gated by a decision does not start before it is closed.

ID	Decision	Options (default first)	Gates
D-01	Policy evaluator	Decided: our own evaluator for our own policy format (Section 1.4.2). Upstream engines are studied for ideas only (D-03).	R1-A
D-02	Object storage	Decided: our own object storage. R1 is a single-node store on local disk with the catalog in PostgreSQL; replication comes later and the S3-compatible API in R3. Everything runs locally for now.	R1-C
D-03	Upstream study and license record	Decided in principle: MinIO and Cerbos are studied for ideas only, and no upstream code is copied, translated or reproduced (SRC-001). The license record is completed before first distribution (SRC-004).	P0
D-04	Identity providers and workload identity	Supported OIDC providers; how workload certificates are issued, rotated and bound to roles (TRU-002).	R1-A
D-05	PostgreSQL HA and failure domains	HA controller, quorum-commit configuration and the physical failure-domain topology (AVL-001).	R1-C
D-06	Label order and label sources	Per-tenant classification order; who may label and which classifier workloads are trusted (LAB-002).	R1-C
D-07	First downstream connector	The one real tool provider used to qualify the executor (SMALL-003).	R1-B
D-08	KMS and seal/unseal mechanism	Reference provider, seal mechanism and recovery custody (KEY-005).	R2
D-09	Key hierarchy	Flat per-unit wrapping is the baseline. The two-level hierarchy (KEK per document version, KEY-001) is adopted only if the sizing measurements (SIZ-001) show a benefit that outweighs its wider compromise impact.	R2
D-10	Limits and S3 client set	Tenant workload limits, object-size limits, and the conformance client list (S3P-005).	R3
D-11	Name and distribution license	Product name, license of original code, license compatibility of dependencies (SRC-004).	first release

0.7 Conventions

Requirements have the form **[ID]** statement (phase · tests). Each states one obligation. The phase is one of P0, R1-A, R1-B, R1-C, R2, R3. The tests are identifiers from Section 6.6.1; `REV` means a release review item and `LINT` means the specification check. A superscript NEW marks a requirement that is new in version 2.0; Appendix E maps every version 1.3 requirement to its successors.

MUST, **MUST NOT** and **SHOULD** carry their usual normative meaning. Limits and numbers marked “initial” or “target” are proposals to be qualified, not measured properties. Sizes use KiB = 1024 bytes. Money uses integer minor units with an explicit currency (DAT-005).

Test vectors in `vectors/` and Appendix B define concrete inputs and expected outcomes. They are part of the specification (TEST-003).

1 Decision core

Part 1 defines who is acting, on what, how a decision is made, how it is bound to an operation, and how the authority state is released, stored, audited and kept available. It does not depend on storage, documents or encryption.

1.1 Identity and trusted context

[ID-001] Identity comes only from verified OIDC or OAuth tokens, workload certificates, signed S3 credentials, or another explicitly registered authentication provider. Issuer, audience, signature, expiry and intended use are validated. (R1-A · T-03)

[ID-002] The platform **MUST NOT** accept an agent- or caller-supplied role, tenant, delegation, device state or user identifier as authority. (R1-A · T-03)

[ID-003] Adapters submit references and operation facts. The context resolver loads roles, ownership, labels, grants, session limits and approved external signals from authoritative state. (R1-A · T-03)

[ID-004] Every trusted context field records its source, observation time, maximum age and revision where available. A missing or stale mandatory envelope fact (verified identity, tenant binding, delegation, version binding) rejects the operation. (R1-A · T-03)

[ID-005] External lookups complete before evaluation under a bounded deadline. Policy expressions **MUST NOT** fetch data or invoke tools. (R1-A · T-03)

[ID-006] Managed-device and recent-authentication facts are usable by policy only when a trusted provider supplies them. A declared purpose may be recorded and constrained but **MUST NOT** be inferred as intent. Network origin is trusted only through configured proxies that strip and regenerate forwarding headers. (R1-A · T-03)

[ID-007] An ID token **MUST NOT** be accepted as an API access token. A JWT access token is accepted only under the resource server's explicit token profile, and opaque tokens only through the provider's supported introspection flow. One JOSE implementation serves all token and envelope signature checks. (R1-A · T-03)

1.2 Delegation

The **actor** is the executing workload or user. The **subject** is the user or service whose authority it exercises. An agent acting for a person carries both; neither inherits the other's full rights.

[DLG-001] A delegation record binds actor, subject, audience or executor, resource and action scope, expiry, parent delegation and an optional budget. (R1-A · T-03)

[DLG-002] A child delegation can only narrow its parent. Delegation depth is bounded; cycles and unknown parent revisions are rejected. (R1-A · T-03)

[DLG-003] The business-policy principal is the represented subject. Without a delegation the authenticated actor is the principal. Actor and subject role sets **MUST NOT** be merged. (R1-A · T-01, T-03)

[DLG-004] The actor's execution capability and the delegation ceiling are validated independently of business policy. Effective authority is the intersection of actor capability, delegation scope and subject permission. (R1-A · T-03)

[DLG-005] Revocation advances a delegation generation that is checked at admission and at commit. (R1-A · T-03, T-13)

[DLG-006] A static API key shared by several agents does not identify the user behind each call. A connector using one **MUST NOT** claim user-specific enforcement without a verified delegation channel. (R1-A · T-03)

[DLG-007] Default credentials carry no implicit administrator or cross-tenant grant. Cross-tenant access requires an explicit, reviewed sharing construct and **MUST NOT** result from a missing tenant filter. (R1-A · T-03)

1.3 Resources, actions and canonical identity

A **check** is one action on one resource binding. A compound operation is several checks evaluated in one batch, all of which must succeed (RES-006).

Resource type	Example actions	Protected facts	Phase
`agent.tool`	discover, invoke, configure	Registry identity, schema digest, connector, capability class	R1-B
`network.destination`	connect, send	Approved hostname or service identity, port, destination class, clearance	R1-B
`business.record`	read, update, approve, export	Organization, owner, workflow state, sensitive fields	R1-B
`document.version`	read-units, raw-read, discover	Canonical document and version, immutable manifest, full-content coverage	R1-C
`document.text_unit`	read, use-in-model, export-key	Canonical unit and object version, label, lineage	R1-C
`document.derived`	deliver, declassify, relabel	Lineage, effective label, intended recipient	R1-C
`text.entry`	read, use-in-model, update	Versioned text unit, parent collection	R1-C
`storage.object`	read, stat, discover, new-version, delete	Tenant, owner, project, label, immutable version, publication state	R1-C
`storage.bucket`	create-object, list, manage	Retention and encryption requirements, namespace restrictions	R1-C
`ai.model`	submit, configure	Approved provider and model, endpoint, credential binding	R1-C
`platform.policy`	propose, test, approve, activate	Bundle digest, reviewer, tenant scope, activation generation	R1-A
`crypto.recipient`	enroll, activate, receive-key, revoke	Principal or workload binding, immutable key fingerprint, purpose, status	R2

Resource type	Example actions	Protected facts	Phase
`crypto.envelope`	create, release, rewrap, retire	Object, tenant and version binding, wrapping-key reference, status	R2
`storage.replica`	repair, inspect, retire	Blob identity, node incarnation, placement generation	R3

[RES-001] Every protected operation maps to one or more (resource, action) checks from the registry. An operation with no registered mapping fails closed. (R1-A · T-01, T-06)

[RES-002] A canonical resource registry maps S3 bucket, key and version, MCP URI and tool selectors to the same tenant-scoped resource UUID, immutable content version, policy kind, version and scope, and trusted attributes. Role membership and principal context resolve through one identity pipeline. (R1-A · T-06)

[RES-003] Content reading uses the canonical action `read`. Legacy route names such as `storage.object.read` are aliases, not competing policies. A caller **MUST NOT** obtain a less restrictive policy kind or scope by changing protocol. (R1-A · T-01, T-06)

[RES-004] Alias resolution happens once under a declared catalog snapshot and is bound to the operation. The resource generation is revalidated before admission and publication. "Latest" **MUST NOT** be resolved again after authorization. (R1-A · T-06)

[RES-005] A rename changes a namespace alias, not the resource's permission identity. An edit creates a new immutable version. (R1-A · T-06)

[RES-006] A compound operation requires a separate allowed check for every resource and action it actually needs. One allow **MUST NOT** authorize an unrelated effect, and `tool.invoke` alone **MUST NOT** authorize reading the tool's source object or sending to its destination. (R1-A · T-06)

[RES-007] A channel difference affects a decision only through an explicitly declared, trusted context attribute that the policy deliberately uses. Authentication or unsupported-protocol failures stay separate restrictions and **MUST NOT** turn a business denial into an allow. (R1-A · T-01, T-06)

[RES-008] Parent-resource checks configured for a content family apply identically through every transport as explicit compound checks, not as a hidden change to scope inheritance. (R1-A · T-06)

1.4 Policy evaluation

1.4.1 Evaluator contract

The decision service calls the business evaluator through a narrow contract so that the engine behind it (decision D-01) can be native or upstream.

```
// Logical contract. Pure: no I/O, no ambient state, time passed in.
EvaluatorRequest {
  snapshot_digest,
  policy_set: BUSINESS | CEILING,
  principal: {id, roles, attrs},
  resource:  {kind, id, version, scope, attrs},
  actions:   [string],
  aux:       {envelope facts mapped as documented attributes},
  now
}
EvaluatorResult {
  per_action: [{action, effect, outputs, metadata, diagnostics}],
  errors:     [{action, kind}]          // strict mode: affected action is DENY
}
```

[**POL-001**] Business policy evaluation sits behind the evaluator contract. The conformance corpus (TEST-003) applies to any engine that implements the contract. (R1-A · T-01)

[**POL-002**] The evaluator implements the project's own policy format and evaluation rules of Section 1.4.2. A document or construct that the format does not define is rejected at bundle validation and **MUST NOT** be silently ignored. (R1-A · T-01, T-02)

[**POL-003**] Evaluation proceeds in this order: (1) authenticate and validate the execution envelope; (1b) evaluate the ceiling set for every check, as an always-run gate that does not depend on the business result; (2) normalize resource, action and principal input and resolve the scope chain; (3) evaluate the business set by the rules of Section 1.4.2, in which a matching deny always overrides an allow; (4) apply the business result unchanged to that check and combine it with the ceiling result so that a ceiling `DENY` always wins; (5) fulfil platform prerequisites and obligations. Platform steps add rejections only and **MUST NOT** turn a business denial into an allow. (R1-A · T-01)

[**POL-004**] The response records the business decision and any platform rejection reason separately. A protocol or authentication failure **MUST NOT** be reported as a policy-combination rule, and no adapter implements alternate granting logic. (R1-A · T-01)

[**POL-005**] Evaluation is strict. An error in the condition of any candidate rule for a requested action yields `DENY` for that action, or an objection in the ceiling set. The format has no lenient mode, so no configuration can select one. (R1-A · T-02)

[**POL-006**] A missing business attribute is handled by the production profile and **MUST NOT** fall through to an allow. Trusted-envelope facts never depend on business-schema settings: invalid credentials, wrong tenant or resource binding, unavailable authority, malformed transport, process faults and exhausted limits always reject. (R1-A · T-02)

[**POL-007**] Platform ceilings are a separate, deny-only ceiling set in the signed bundle. Every check is also evaluated against the ceiling set through the evaluator contract (`policy_set: CEILING`) before and independently of the business evaluation, and a ceiling `DENY` or a ceiling evaluation error denies the check whatever any principal, role, derived-role or scope result says. Bundle validation rejects a ceiling set that contains a principal policy, an allow rule or a scope override. (R1-A · T-01, T-02)

[**POL-008**] Time is captured freshly for each request or batch and never frozen at bundle preparation. Admission and commit checks use current server time. (R1-A · T-01)

[**POL-009**] Policy expressions cannot fetch network data, read files, execute scripts or acquire credentials. Compilation, input size, evaluation cost, memory and concurrency are bounded independently of business-policy options. (R1-A · T-02)

[**POL-010**] A snapshot is prepared once per immutable bundle. It contains the validated policy graph, schemas, checked executable expressions, configuration and dispatch indexes keyed by

static resource, action, version and scope facts. The index **MUST NOT** omit a rule because another path looks permissive. A compiled expression is shared only when the complete environment identity (types, function signatures, schema digests, library and options versions, cost model, evaluator-semantics version) matches. (R1-A · T-01)

[**POL-011**] Policy outputs are data. A mapping from selected output fields to platform obligations is allowed only when explicitly configured, versioned and validated. No mandatory proprietary policy field is introduced for approvals or storage behavior. (R1-A · T-01)

[**POL-012**]NEW The conformance corpus of policy-evaluation vectors is part of this specification and runs against the engine in CI. No upstream engine is used as a reference or oracle (D-01). (R1-A · T-01)

[**POL-013**] YAML authoring is converted offline into the same strict typed representation as JSON. Ambiguous or unsafe constructs are rejected, and evaluation never runs a YAML parser. (R1-A · T-02)

[**POL-014**]NEW Scopes form a dot-separated chain from the request's scope to the root. Rules from every scope on the chain are combined by the same deny-overrides rule, so a child scope can add allows but cannot lift a deny from a parent scope or from the root. (R1-A · T-01)

1.4.2 Policy format

The policy format is the project's own (D-01). A document holds rules for one resource type in one scope and one policy set:

```
{
  "format": "fga.policy/v1",
  "set": "business",
  "resource": "document.text_unit",
  "scope": "acme",
  "rules": [
    {"id": "reviewers-read-own-projects",
     "effect": "allow",
     "actions": ["read"],
     "roles": ["reviewer"],
     "when": "resource.attributes.project_id in principal.attributes.project_ids"}
  ]
}
```

The ceiling set uses the same document shape with `"set": "ceiling"` and deny rules only, for example a rule that denies `read` for any role `when` the unit is not LIVE.

For one principal, resource and action, the engine:

1. collects the candidate rules of the requested set for the resource type from the request's scope and each ancestor up to the root;
2. treats a rule as matching when it lists the action (or `*`), the principal holds one of its roles (or `*`), and its condition is true or absent;
3. for the business set, returns `DENY` if any matching rule denies, otherwise `ALLOW` if any matching rule allows, otherwise `DENY`;
4. for the ceiling set, reports an objection if any matching rule denies, and no objection otherwise;
5. denies (or objects) when the condition of any candidate rule fails to evaluate.

Conditions are CEL expressions over `principal` (`id`, `roles`, `attributes`), `resource` (`type`, `id`, `scope`, `attributes`) and `request` (`now`). They are compiled once per bundle and run under a cost limit. Because a deny always overrides an allow, holding an extra role cannot undo a deny, and

no business rule can bypass a ceiling. Principal-specific policies, derived roles, shared variables and outputs are not part of version 1; they can be added later without changing the meaning of existing policies.

1.5 Decision API

```
DecideBatch request {
  request_id, tenant_id, adapter_id, operation_id,
  actor:      {id, type, identity_revision},
  subject:    {id, type, identity_revision},
  delegation: {id, revision},
  session:    {id, binding_revision, context_key,
               mark_revision, label_order_generation},
  checks:     [{check_id, action,
                resource: {type, id, version, generation, metadata_revision}}],
  operation_digest, required_policy_epoch, deadline
}
DecisionResponse {
  outcome: ALLOW | DENY | NEEDS_APPROVAL | NEEDS_RESERVATION | ERROR,
  reason_code, decision_id,
  policy_results: [{check_id, principal_id, resource_id, action,
                    business_effect | NOT_EVALUATED, ceiling_effect,
                    policy_outputs,
                    policy_metadata, diagnostics}],
  bundle_digest, policy_epoch, evaluator_version,
  actor_id, subject_id, delegation_revision,
  operation_digest, resource_bindings_digest, context_revisions,
  evaluated_at, valid_until,
  obligations: [{type, version, parameters_or_ref, parameters_digest}],
  session_mark_revision_after,
  signature                        // detached JWS, see DEC-008
}
```

`Decide` is a batch of one. A platform-gate rejection returns `DENY` with a platform reason code and `NOT_EVALUATED` business effects. `ERROR` means the service could not decide (timeout, unavailable authority, resource limit).

[**DEC-001**] The decision API is exposed as versioned gRPC for internal services and an authenticated JSON facade for SDKs. The caller **MUST NOT** choose an action mapping different from the registered operation. (R1-A · T-01)

[**DEC-002**] The enforcement point acts only on an explicit, complete `ALLOW` bound to its exact operation. `DENY`, `NEEDS_APPROVAL`, `NEEDS_RESERVATION`, `ERROR`, transport errors, malformed data, unknown required obligations and missing results all stop execution. A prerequisite outcome is satisfied through the control plane, after which a new `ALLOW` whose context revisions include the approval or reservation is required. (R1-A · T-01)

[**DEC-003**] Each response carries the per-check business and ceiling effects, outputs, metadata, redacted diagnostics, bundle digest, policy epoch, evaluator version, identity and delegation revisions, operation and resource-bindings digests, validity and obligations. A platform gate that stops evaluation is reported with an explicit not-evaluated marker and **MUST NOT** authorize execution. (R1-A · T-01)

[**DEC-004**] A batch captures one policy snapshot and one authority envelope. Items have unique identifiers and exact resource, action and revision bindings. A missing, duplicated or misbound result **MUST NOT** authorize its operation, and a failed item **MUST NOT** grant its peers. A batch is an evaluation optimization, not a transaction over writes or external effects. (R1-A · T-01)

[DEC-005] `valid_until` is bounded by the earliest of identity or session expiry, delegation expiry, approval or grant expiry, context freshness limit, request deadline and a conservative decision-age limit. A decision is not a general bearer permission. (R1-A · T-13)

[DEC-006] At read admission and immediately before a mutation's irreversible or egress checkpoint, the executor rechecks expiry and every authority-bearing revision recorded with the decision: resource generation; metadata revision; access and label revisions of the parent and of each selected unit; delegation and identity revisions; context mark revision and label-order generation; and the required policy epoch. A mismatch requires a new decision or rejection. The executor uses the same canonical operation that was evaluated. (R1-A · T-13)

[DEC-007] Identity and capability checks are enforced at the decision API itself. mTLS workload identity or an equivalent bound credential identifies the adapter, and an ordinary user cannot submit fabricated verified context. (R1-A · T-03)

[DEC-008][NEW] The decision service signs each `ALLOW` record with a registered decision-signing key, used for no other purpose, whenever the decision will be used across a process or role boundary or retained. The signed bytes are one canonical serialization (strict JSON with no unknown or duplicate fields, signed exactly as transmitted) containing: decision ID; audience (deployment, role and service); tenant; actor, subject, delegation and context identity with their revisions; operation digest; every check with its resource binding and effect; bundle digest and policy epoch; access, label and context-mark revisions and the label-order generation; and issue time and `valid_until`. A verifier checks signature, audience, validity and the replay rule, and still performs the fresh revision checks of DEC-006: a signature does not replace them. A role **MUST NOT** accept a decision whose audience is another role. A signed decision proves that the sole authority evaluated the request it describes; it is not proof that the user's intent or the request's context was validly established. (R1-A · T-03)

[DEC-009] Initial qualification limits are a 250 ms single-decision budget, 64 KiB of context, 50 resources per batch, 8 KiB of expression text, bounded AST depth, 10,000 expression cost units per condition and a total request budget. These are proposed limits, not measured guarantees; independent input, compilation, allocation, concurrency and cancellation limits are mandatory, and cancellation reaches both queued and running evaluation. (R1-A · T-22)

[DEC-010] Error diagnostics distinguish a policy denial from a service failure without exposing protected details to unauthorized clients. (R1-A · T-01)

[DEC-011][NEW] Consumption of a single-use signed decision (mutation and key release) is an atomic, durably acknowledged compare-and-set in the catalog, shared by every replica of the verifying role and recorded before the effect begins. A replica that cannot reach the consumption store refuses the decision. (R1-A · T-03)

[DEC-012][NEW] Decision-signing keys are distinct from workload, issuer and encryption keys. Key identifiers reach verifiers through the control plane, rotation uses an overlap window no shorter than the longest `valid_until` plus clock skew, and a compromised signing key is revoked by a revision that verifiers must observe before accepting further decisions. Rotation and revocation are qualified under T-03. (R1-A · T-03)

1.6 Policy releases and readiness

[REL-001] A policy release is an immutable, signed bundle whose manifest includes schema and evaluator versions, source hashes, the business set, the ceiling set, the label order and its generation, resource schemas, operation registry, pure-function allowlist, expression-library version and cost model, test evidence, target audience and digest. Each instance verifies, parses, compiles, validates and checks a candidate into an immutable in-memory snapshot before atomically replacing its active pointer. A failed activation preserves the prior valid state and does not acknowledge the

candidate. (R1-A · T-14)

[REL-002] Activation records the states RECEIVED, VERIFIED, PREPARED, READY, ACTIVE or REJECTED with digest, environment version, preparation error and timestamps. Every decision uses exactly one snapshot, and preparation and old-snapshot coexistence are resource-bounded. (R1-A · T-14)

[REL-003] A node that cannot establish the authoritative required epoch **MUST NOT** authorize new operations. The last valid snapshot is kept only while it satisfies that epoch. Offline authorization is not an availability fallback. (R1-A · T-13, T-14)

[REL-004] The control plane stages a candidate pool, verifies sentinel allow and deny cases, records ready instances by digest, advances the durable tenant-required epoch, and routes only to compatible instances. A restrictive activation may stop admissions and drain bounded mutation leases. A rollback is a new reviewed release with a higher epoch, because silently reverting may restore revoked access. (R1-A · T-14)

[REL-005][NEW] Before activation the candidate is run over the conformance corpus and the tenant's sentinel cases, and the change in effect against the active release is reported to the reviewer, who must acknowledge it. The report covers the corpus and sentinels only; it does not enumerate every principal. (R1-A · T-14)

[REL-006] Liveness reports process progress and does not restart a healthy process because PostgreSQL, KMS or a peer is unavailable. Readiness is role- and capability-specific (policy digest and epoch, catalog authority, workload credential validity, required crypto and connector dependencies). Probes return redacted aggregate state, are rate-limited, and do not create a KMS key. A readiness result is never a grant; requests perform their own admission checks. (R1-A · T-14, T-19)

1.7 State and catalog

PostgreSQL holds authoritative identities, grants, resource bindings, revisions, approvals, reservations and audit-outbox records.

Entity	Minimum authoritative fields	Phase
Principal, delegation	Stable IDs; tenant; actor and subject bindings; credential class; roles; limits; expiry; revocation and identity revisions	R1-A
Resource	UUID; type; tenant; protected attributes; metadata revision; resource generation; lifecycle state	R1-A
Policy release	Canonical manifest digest; schema, configuration, evaluator and expression-library versions; tests; publisher signature; activation epoch	R1-A
Audit outbox	Unique event ID; decision and operation IDs; redacted bindings; outcome; delivery status	R1-A
Tool registry	Stable tool ID; schema digest; connector version; operation mapper; allowed destinations; required capabilities; credential binding	R1-B
Invocation	Tenant and idempotency unique key; argument digest; state; dispatch attempt and fencing token; downstream receipt; reconciliation status	R1-B
Approval	Exact operation digest; approver; subject and actor; resource revisions; expiry; allowed uses; consumption state	R1-B
Reservation	Budget or grant ID; minor-unit amount or use count; invocation ID; status; lease; reconciliation state	R1-B

Entity	Minimum authoritative fields	Phase
Object, version	Bucket and exact key; stable object UUID; current committed version; content generation; immutable blob ID; length and checksums	R1-C
Document manifest, text unit	Document and version UUIDs; ordered unit IDs; manifest digest; extraction and normalization version; unit order; labels; access revision; provenance	R1-C
Upload intent	Operation and idempotency IDs; principal and delegation; expected generation and current version; intended labels; staged blob; state; expiry	R1-C
Context mark	Context key (runtime context ID, else delegation ID); bound actor, subject and delegation; label-order generation; highest effective label; mark revision	R1-C
Recipient key, key envelope	Tenant; principal or workload; immutable key ID and fingerprint; purpose and suite; activation and revocation revision; wrapped keyset; generation; status	R2
Maintenance job	Kind; tenant and blob scope; expected placement generation; idempotency key; lease and fence; progress; initiating actor	R3

[**DAT-001**] All authoritative lookups include tenant scope. Tenant-consistent references are enforced with database constraints or equivalent transaction checks. Tenant isolation **MUST NOT** depend on policy authors writing a tenant condition. (R1-A · T-01)

[**DAT-002**] Protected fields change only through explicit authorized control operations with expected revisions. Raw S3 tags, tool arguments and external payloads **MUST NOT** become trusted ownership or classification records. (R1-A · T-03)

[**DAT-003**] Resource metadata revision and content or resource generation are distinct. Every logical publication, deletion or target-state mutation increments the generation, and approval and execution comparisons include state changes even when access labels have not changed. (R1-A · T-06, T-13)

[**DAT-004**] Permission-critical facts use authoritative consistent reads. The first release has no allow-result cache. A cache, if added, requires a maximum age, revision-aware keys and explicit invalidation. A decision uses one immutable policy snapshot and one declared context snapshot. Search indexes and storage notifications are discovery aids, not access authority. (R1-A · T-13, T-14)

[**DAT-005**] Money uses integer minor units with an explicit currency. Binary floating point **MUST NOT** be used for permission thresholds or budget consumption. (R1-B · T-05)

1.8 Audit

[**AUD-001**] Decisions and actual outcomes are logged separately and joined by decision, operation and invocation IDs. Records include actor, subject, tenant, action, resource, version and generation, relevant context provenance, policy digest and epoch, reason, latency and redacted obligations. External tools record dispatch and receipt or reconciliation evidence. (R1-A · T-15)

[**AUD-002**] Attempt, decision, key release, durable mutation, approval consumption and external delivery are distinct events. Mandatory records are appended transactionally to an outbox and **MUST NOT** silently disappear when the exporter is disabled or a queue is full; capacity alarms and declared admission limits act before required evidence can no longer be recorded. Operational logs and optional notifications are not a substitute for the outbox. (R1-A · T-15)

[AUD-003] Audit delivery is idempotent with explicit retention and access policies. Sensitive labels and arguments are minimized or protected, lag, loss, stale policy nodes, failed context resolution and revocation latency are monitored, and metrics **MUST NOT** create a cross-tenant information channel. (R1-A · T-19)

[AUD-004] A key-release audit record proves that a key was issued, not that a user opened plaintext or complied with a purpose restriction. Documentation **MUST NOT** present it as proof of use. (R1-A · REV)

1.9 Administration

[ADM-001] Policy authors, reviewers, activators, key administrators, storage operators and data users are separate roles. A policy administrator is not automatically a data reader, and sensitive policy or key changes use the approval and reservation model where configured. (R1-A · T-20)

[ADM-002] An ordinary API request **MUST NOT** disable the authorizer, change trusted issuers, relax workload audience checks, replace a key provider or grant itself policy-administrator authority. Those changes use explicit administrative actions and reviewed control transactions. (R1-A · T-20)

[ADM-003] Bootstrap and recovery authority sits outside ordinary agent paths, with controlled local or separately authenticated recovery and a durable incident audit. Recovery procedures **MUST NOT** claim to defend against a host operator who can replace the running software. (R1-A · REV)

1.10 Availability of authority state

This section covers the decision, control and catalog services. Blob replication is in Part 5.

Layer	Initial production topology	Failure contract
Gateways, decision, tools	At least 3 stateless replicas across 3 failure domains, required bundle digest on each	Load balancer removes unready replicas; stale or unavailable authority never becomes allow
Tool executors	At least 3 workers sharing durable invocation state	Leases and fencing plus downstream idempotency; no blind redispatch after timeout
Catalog PostgreSQL	Primary plus 2 standbys; commit durable on the primary and at least one synchronized remote standby (quorum commit)	HA coordinator promotes only a safe candidate and fences the old primary; no silent asynchronous downgrade

[AVL-001] Production runs the topology above. Single-node operation is a development profile and is rejected by production configuration. (R1-C · T-15, T-16)

[AVL-002] Authorization-critical reads use the primary or a standby proven sufficiently fresh by a log-position check. An arbitrarily lagging standby **MUST NOT** restore revoked grants. (R1-C · T-16)

[AVL-003] PostgreSQL runs with `fsync=on`, `full_page_writes=on`, `synchronous_commit=on` or stronger, and a non-empty synchronous standby set; `remote_write` alone is insufficient against an operating-system crash. With Patroni, strict synchronous mode is enabled. Failover uses a

managed HA service or established controller with quorum coordination and old-primary fencing, and promoting an unqualified stale replica voids the zero-loss objective. (R1-C · T-16)

[**AVL-004**] A timeout or cancellation while awaiting a synchronous commit is an unknown outcome, and a lookup by operation ID on the primary is not evidence of durability: after a cancelled wait the transaction can be visible on the primary without having been replicated. Before returning durable success, dispatching an external effect or releasing a key, the platform obtains proof that the required synchronous standbys have flushed WAL at or beyond the transaction's commit position on the current fenced timeline, either from an acknowledged synchronous commit ordered after it or from the replication flush position. Until then the outcome stays unknown. (R1-C · T-16)

[**AVL-005**] The design target is zero loss of acknowledged operations under the loss of one failure domain in a crash-fault model, conditional on synchronous database acknowledgement, correct fencing and available key state; it is not a guarantee against simultaneous multi-domain loss, Byzantine replicas, malicious deletion or untested hardware. Recovery of the target service within five minutes is an acceptance objective, not a benchmark. Encrypted, access-separated backups of the catalog, policy bundles and (from R2) key envelopes and KMS recovery state are restored together in tests, and replication is not a substitute for backups. (R1-C · T-15, T-18)

[**AVL-006**] Tenant-aware admission limits cover requests, bytes, active streams, expression cost, scan candidates, reservations, tool concurrency and KMS operations. Overload uses backpressure and bounded queues rather than unbounded goroutines, and retries use jitter and budgets. Circuit breakers reject rather than grant. (R1-C · T-22)

[**AVL-007**] The first metadata authority is one PostgreSQL primary with HA. When measured write capacity is exceeded, tenants are sharded across independent catalog groups with an explicit tenant-to-shard registry and migration fencing, and cross-shard grants and budgets are not treated as local atomic operations. The first region is active for each tenant; multi-region active-active authorization requires a later consistency design. (R1-C · T-22)

2 Enforcement points

Part 2 defines how a decision is bound to a real operation: the adapter contract, the evidence that crosses each internal boundary, the tool executor, approvals and reservations, unknown outcomes, data-flow limits, and the MCP transport. Content-specific MCP operations are in Part 3.

2.1 Adapter contract

```
// Logical design; generated DTOs are defined by the versioned API.
type EnforcementAdapter interface {
    Prepare(ctx context.Context, request AdapterRequest) (OperationPlan, error)
    Execute(ctx context.Context, plan OperationPlan, grant Grant) (Outcome, error)
}

type OperationPlan struct {
    ID              string
    RegistryVersion string
    RequiredChecks  []Check
    ArgumentsDigest string
    ResourceBindings []ResourceBinding
    IdempotencyKey  string
}
```

[**ENF-001**] Every adapter implements `Prepare` and `Execute`. `Prepare` resolves targets and validates arguments without performing the protected side effect. `Execute` verifies the grant, carries

out the required obligations and dispatches only the prepared plan. (R1-B · T-04)

[ENF-002] An adapter request is a validated typed protocol or domain variant. It **MUST NOT** be an unrestricted `any` or an arbitrary attribute map, and each concrete adapter accepts only its declared request kind. (R1-B · T-04)

[ENF-003] An adapter translates authentication evidence, selectors, operation parameters and response framing. It **MUST NOT** fork business grants, duplicate policy combination, invent its own context format, turn an unknown policy result into a local allow, or perform a permissive check and then bypass the shared service. (R1-B · T-01, T-04)

[ENF-004] Where target resolution is itself sensitive, it requires a separate discovery or stat action. (R1-B · T-04)

[ENF-005] Every adapter declares its discovery, read, mutation, output-transformation, batching, streaming and obligation capabilities. An unknown capability fails closed. Public API clients receive redacted errors while internal audit retains diagnostic detail, and the resource-to-action mapping is versioned with policy and schema releases. (R1-B · T-04)

[ENF-006] Database row or column filtering, filesystem mounts, message consumption and job scheduling are extension points that need real enforcement at their own access boundary. The first release does not compile expressions into SQL; a later optimizer must translate a declared subset, prove equivalence against per-resource decisions and never drop an unsupported predicate. A database adapter exposes a field only if it actually builds a restricted query or response. (R1-C · T-04)

2.2 Trust propagation

Authentication at an ingress is insufficient if a downstream service can be reached another way. Each internal call carries the evidence in Table 10, and the receiver rejects a call that lacks it.

Table 10: Evidence that crosses each boundary

TB	Hop	Evidence the receiver verifies	Phase
TB1	Client → adapter	Token or credential for this audience (ID-001); session bound to actor, subject and delegation (MCP-007)	R1
TB2	Adapter → decision	Workload certificate bound to the adapter role (DEC-007); references, not asserted facts (ID-003)	R1-A
TB2	Decision → adapter	Signed decision for this audience, operation and validity (DEC-008)	R1-A
TB3	MCP adapter → content service	Workload certificate of the `tools` role, plus signed decision or a delegation reference verified against authoritative state (TRU-003)	R1-C
TB3	Content service → broker	Workload certificate of the `storage` role plus a signed decision whose checks are exactly those of the matching operation mapping of Table 15, bound to the exact object, version, unit and recipient (TRU-005)	R2
TB4	Executor → connector	Target-specific restricted credential, never the model's credential (AGT-006)	R1-B

TB	Hop	Evidence the receiver verifies	Phase
TB4	Executor → downstream	Idempotency key and state precondition where supported (AGT-006, INV-005)	R1-B
TB5	Storage → blob backend	Verified workload binding on every call (BKD-001)	R1-C

[TRU-001] Each cross-process hop carries the evidence listed in Table 10. A receiver that does not find it rejects the call. (R1-A · T-03)

[TRU-002] Workload identity uses mTLS or an equivalent bound credential issued by the workload certificate authority chosen in decision D-04, bound to one role. Certificate verification is mandatory configuration, not an optional check. (R1-A · T-03, T-20)

[TRU-003][NEW] The end user's authority crosses a hop only as a signed decision (DEC-008) or as a delegation reference that the receiver verifies against authoritative state. Caller-asserted claims plus a workload identity are never sufficient. (R1-A · T-03)

[TRU-004][NEW] A role accepts only signed decisions whose audience is that role, and the decision-signing key is registered separately from workload, issuer and encryption keys. (R1-A · T-03)

[TRU-005][NEW] The broker releases key material only against a signed decision whose checks are exactly those of the matching operation mapping in Table 15 and that is bound to the exact object, version, unit and recipient being released. The broker verifies the complete signed result, not a subset, whether or not the request also carries an mTLS identity. A compromised gateway therefore cannot obtain keys that no valid decision covers. (R2 · T-11)

[TRU-006] The MCP adapter calls the same content-service implementation as every other adapter and preserves the represented principal and delegation. It **MUST NOT** replace the user with a broadly privileged connector account. (R1-C · T-03)

2.3 Tool executor

The agent proposes an invocation. A trusted executor authenticates it, resolves the registered tool, validates arguments, computes the real operation plan, obtains authorization and performs the effect with credentials the model never sees.

[AGT-001] A registry record binds tool ID, version, schema digest, connector implementation, resource mapper, allowed destinations, required checks and supported obligations. Tool descriptions and annotations are untrusted unless supplied by the registry, and a label such as "read-only" does not establish behavior. A changed schema digest invalidates the registration until re-reviewed. (R1-B · T-04)

[AGT-002] Hiding a tool from discovery improves usability but is not a security boundary. Every direct invocation is checked. (R1-B · T-04)

[AGT-003] The executor performs these steps in order and **MUST NOT** skip or reorder them. (1) Validate actor identity, represented subject, tenant, audience-bound session and delegation, and reject on-behalf-of headers supplied by the caller. (2) Resolve the tool and version and validate arguments against a strict schema, rejecting unknown fields where security depends on exact interpretation. (3) Map the request to concrete resources and actions (for example a submission, an account, a currency and amount, and a destination service), canonicalize them and compute the operation digest. (4) Evaluate every required check, resolve approval, reservation, destination restriction and other obligations before execution, and bind them to the exact plan. (5) At dispatch recheck expiry, generations, delegation and target binding, and use a target-specific restricted credential. (6) Execute once under the invocation record, and record success, denial, failure or

unknown outcome. (R1-B · T-04)

[AGT-004] No mutation between approval and dispatch may change tool version, target, resource, amount, destination or another security-relevant argument. Canonicalization is shared by authorization and execution. (R1-B · T-04)

[AGT-005] The executor **MUST NOT** validate one URL and then follow an arbitrary redirect to another destination. Address policy is resolved at connection time and credentials are bound to the registered service identity. (R1-B · T-04)

[AGT-006] The connector uses a target-specific restricted credential, never the model's proposed credentials. State and version preconditions (ETag, version ID or transactional compare-and-swap) are passed downstream where authorization depends on target state, and a connector without that capability **MUST NOT** claim state-bound mutation safety. (R1-B · T-04)

[AGT-007] HTTP MCP tokens are validated for the gateway's audience and **MUST NOT** be passed through to downstream APIs. The connector obtains its own limited downstream token; token-exchange and delegation profiles are tested rather than assumed. (R1-B · T-03)

[AGT-008] Local stdio tools run through the trusted executor and an operating-system sandbox with bounded filesystem, process and network access. Tool servers are network-restricted so users cannot bypass the gateway with the same credentials. A remote policy service cannot intercept an unrestricted local shell or direct local file reads, and the documentation says so. (R1-B · T-20)

2.4 Approvals and reservations

A pure policy decision can establish eligibility but cannot atomically consume a budget. Approvals, one-time grants and spending limits require stateful transactions in PostgreSQL.

[APR-001] A prerequisite approval or reservation is satisfied before execution. An approval binds the exact operation digest, actor and subject, target, relevant resource revisions, policy epoch, expiry and permitted uses, and a change to any material fact invalidates it. (R1-B · T-04)

[APR-002] Approver authority and the prohibition on self-approval are checked separately. Approval consumption and reservation are atomic, and a model-generated assertion **MUST NOT** stand in for the approval record. (R1-B · T-05)

[APR-003] An approval is consumed at the documented execution checkpoint, not when it is merely displayed. (R1-B · T-05)

[APR-004] A reservation cannot exceed its parent grant or budget under concurrency. Capacity is reserved under row-level locking or serializable transactions with unique operation and idempotency keys. (R1-B · T-05)

[APR-005][NEW] The summary shown to an approver is rendered by the platform from the canonical operation plan (tool, target, amounts, destinations, resource list and highest source label). Agent- or model-supplied descriptions may be displayed only as clearly labeled unverified notes. (R1-B · T-04)

[APR-006] Cancellation, expiry, outcome-unknown and compensation are distinct states. (R1-B · T-05)

[APR-007] Obligations from several sources merge restrictively: field and destination sets intersect, required redactions union, and limits take the minimum. Conflicting or unsupported obligations reject, and obligations are capability-checked and enforced before their effect or disclosure. (R1-B · T-04)

2.5 Invocation state machine and unknown outcomes

The states are PREPARED → RESERVED → DISPATCHING → SUCCEEDED, FAILED or UNKNOWN. A network timeout after dispatch is not proof that the external operation failed.

[INV-001] Invocation state transitions are recorded durably and each transition has one writer under the invocation lease. (R1-B · T-05)

[INV-002] A unique invocation row and a transactional lease with a monotonically increasing fencing token give exactly one worker dispatch ownership. Approval consumption and budget reservation commit with the ownership transition, and in the production topology that transition passes the durability barrier of AVL-004 before the external dispatch begins. (R1-B · T-05)

[INV-003] A replacement worker reconciles a DISPATCHING or UNKNOWN invocation and **MUST NOT** redispatch it blindly. (R1-B · T-05)

[INV-004] A retry of the same operation uses the same idempotency identity and exact argument digest. Changing arguments starts a new operation that requires new authority. Read-only calls may use a separate bounded retry policy. (R1-B · T-05)

[INV-005] Retrying a mutation requires downstream idempotency support or target-specific reconciliation. No exactly-once promise is made across arbitrary external APIs, and tool-worker fencing cannot stop an already-running remote request unless the downstream enforces an idempotency, fencing or precondition token. (R1-B · T-05)

[INV-006][NEW] An UNKNOWN invocation is resolved by connector reconciliation, or by an `invocation.resolve` action that records evidence and one of three outcomes: success, failure verified by evidence, or abandoned (INV-009). The tool's maximum hold age triggers escalation and alarms (at half and at the full age) and never infers failure. For financial or destructive effects the resolver **MUST NOT** be the original requester. (R1-B · T-05)

[INV-007] A reservation for an operation with an unknown external outcome **MUST NOT** be released as though no effect occurred. It remains until reconciliation or resolution commits or releases it. (R1-B · T-05)

[INV-009][NEW] ABANDONED closes the workflow without establishing that the external effect did not happen. It keeps the reservation charged, keeps the approval consumed, prohibits redispatch of the operation and retains the evidence. A reservation is released only by a verified failure or by a separate `reservation.write-off` action, authorized to a principal other than the requester, that records the accepted risk. (R1-B · T-05)

[INV-008] After an ambiguous failover, connectors that cannot reconcile remain UNKNOWN. The same rule applies to budget accounting and approval consumption. (R1-B · T-05)

2.6 Data flow and the context high-water mark

[FLW-001] Read permission does not constrain what a model subsequently infers, says or sends. Enforcing destination restrictions requires mediation of all relevant egress, file writes and downstream tool calls; if the model has an unrestricted independent network path, the deployment **MUST NOT** claim complete output-flow enforcement. (R1-C · T-21)

[FLW-002] A tool that reads confidential storage and sends data requires both source-read and destination-send permission, executed through one prepared plan. Raw storage and KMS credentials are not exposed to the model. (R1-C · T-21)

[FLW-003][NEW] For every mediated execution context the platform records the highest effective label of any unit or derived output disclosed to it (tool results, resource reads, model context). An egress-capable call whose destination clearance is below the mark is `DENY`, or `NEEDS_APPROVAL` when a policy output says so. The mark only increases within its context. (R1-C · T-21)

[FLW-004][NEW] Labels form a tenant-defined total order in the signed bundle, identified by an order generation, and the order is immutable within a generation. A unit or output with no label counts as the highest level. A mark records the generation it was computed under; when the active generation changes, a mark from an older generation is treated as the highest level unless the bundle supplies a verified monotone mapping from the old order to the new one. (R1-C · T-21)

[FLW-005][NEW] The mark is conservative: it cannot tell which disclosed content influenced a call, so it over-restricts. It does not cover paths outside mediated tools, or runtimes that discard their context lifetime, and product documentation states those limits. (R1-C · T-21)

[FLW-007][NEW] A mark raise is an atomic, monotonic update that is durably acknowledged before any byte of the disclosure is emitted. The decision request carries the mark revision and label-order generation, and the signed decision binds them. At egress dispatch the executor rechecks the mark revision under the invocation lease and rejects the dispatch, requiring a new decision, if the mark has risen since. Once an external call or stream is admitted and dispatched, a later raise does not retract it. (R1-C · T-21)

[FLW-008][NEW] The mark belongs to a trusted execution-context lifetime. It is keyed by a context ID from a registered runtime when one is supplied, and otherwise by the delegation ID. A new MCP connection or session ID under the same key inherits the mark, and only a new key starts at the lowest level. A mark is reset early only by an explicit, audited, approval-gated `context.reset` action. The guarantee holds for runtimes that preserve their context lifetime (FLW-005). (R1-C · T-21)

[FLW-006] Classification propagation and output inspection beyond the mark are separate mechanisms with explicit false-negative and false-positive limits. “Use only for purpose X” cannot be guaranteed by an ordinary boolean policy or by encryption alone. (R1-C · REV)

2.7 MCP transport and surface

[MCP-001] The MCP endpoint uses the official MCP Go SDK for protocol transport and JSON-RPC machinery. Project code supplies the trusted registry, policy mapping and execution boundary; a hand-written transport is not the default. (R1-C · T-06)

[MCP-002] MCP has no independent permission rules, document store, user database or evaluator. Its handlers translate requests and responses and delegate to the shared content and decision paths. (R1-C · T-06)

[MCP-003] Discovery and invocation are checked separately from content access. `tools/list` or a permit for `documents.read` as a tool does not authorize every document. `resources/read` and every content-returning tool resolve the exact resource, authorize the parent and units, and only then obtain and return permitted plaintext. A connector that cannot identify the protected content before disclosure is rejected or restricted to a fully authorized whole-resource profile, and an opaque upstream tool result is not assumed filtered. (R1-C · T-06, T-09)

[MCP-004] Document and tool content is untrusted data, never a source of roles, permissions, trusted key locations or execution authority. Retrieved instructions cannot change the tool registry, broaden a delegation, choose a new provider endpoint or grant a follow-up tool call, and the executor checks each new effect under the same contract. (R1-C · T-03, T-04)

[MCP-005] Standard MCP clients receive authorized plaintext or resource links. HPKE ciphertext is returned only to clients that explicitly support the export profile. (R2 · T-10)

[MCP-006] Where MCP runs in a different role process, its authenticated RPC client calls the same service implementation and preserves the represented principal and delegation. (R1-C · T-06)

[MCP-007][NEW] An MCP or tool session identifier is bound to the authenticated actor, subject and delegation, and its binding revision is part of the decision request. A session identifier alone is never

authority and does not by itself define the lifetime of the context mark (FLW-008). (R1-C · T-21)

2.8 HTTP, service and future adapters

[ADP-001] HTTP middleware authenticates the caller, resolves the route and resource, validates the request body and checks the exact semantic action immediately before protected work. Path parameters and request bodies are not authoritative ownership records. (R1-C · T-06)

3 Content, documents and disclosure

Part 3 defines the single content service, how content is published over a storage backend, how documents are split into independently authorized units and labeled, how units are disclosed to people, tools and models, and how derived outputs are controlled. In R1 the backend provides at-rest encryption; per-unit cryptographic separation is R2 (Part 4).

3.1 Content service

```
// Logical application interface; all callers retain verified identity context.
type ContentService interface {
    Read(ctx context.Context, req ReadContentRequest) (ContentReader, error)
    Stat(ctx context.Context, req StatContentRequest) (ContentMetadata, error)
    List(ctx context.Context, req ListContentRequest) (ContentPage, error)
    Write(ctx context.Context, req WriteContentRequest) (PublishedVersion, error)
    Delete(ctx context.Context, req DeleteContentRequest) (DeleteOutcome, error)
    Export(ctx context.Context, req ExportContentRequest) (ExportHandle, error)
}
```

[CNT-001] `internal/content` is the single application layer for content reads, stat and discovery, listing, writes, deletion and authorized export. HTTP, MCP and (in R3) S3 handlers delegate to it through typed calls. It resolves canonical aliases and versions, performs the business checks and the mandatory execution guards, and then uses the storage and key services. (R1-C · T-06)

[CNT-002] Protocol adapters translate authentication evidence, selectors, operation parameters and response framing. They **MUST NOT** perform a permissive content check and then bypass the shared service. (R1-C · T-06)

[CNT-003] A whole-document or original-file path authorizes its complete disclosure set. A parent allow or a permitted section cannot authorize a forbidden section through raw download, preview, export or an answer that quotes it. Original-file delivery requires an authoritative complete-content manifest and all corresponding decisions; unsupported or unclassified original content rejects that path. Authorized-subset projection is explicit and does not silently alter what an ordinary GET returns. (R1-C · T-06)

[CNT-004] HEAD and error responses **MUST NOT** reveal existence or metadata to callers that lack stat or discovery authority. (R1-C · T-09)

[CNT-005] A listing authorizes bucket listing, scans committed catalog candidates in stable key order, batch-checks per-object discovery, and accumulates visible entries until the page size or a bounded scan or time budget is reached. A short or empty page may carry a continuation token and the cursor must make progress. Metadata is filtered as well as names. A continuation token binds tenant, principal, identity revision, query, prefix and delimiter, policy epoch, expiry and last scanned position; its contents are encrypted and authenticated or held in opaque server state, and replay under another identity or a changed query fails. `CommonPrefixes` **MUST NOT**

expose hidden children, and delimiter support stays disabled until safe prefix discovery passes tests. (R1-C · T-09)

[CNT-006] Listings and discovery are read-only from the caller's perspective. They may report a bounded integrity hint but **MUST NOT** execute expiration, deletion, declassification or replication as a hidden side effect; background maintenance owns those effects under its own authority and job record. (R1-C · T-09)

[CNT-007][NEW] Candidate selection for listing and search may use a sound superset prefilter (for example tenant and project predicates evaluated in SQL). Every candidate that survives still receives an exact per-resource decision, and a prefilter is never the only check. (R1-C · T-09, T-22)

[CNT-008] A read revalidates decision and session expiry and revisions before response admission. An admitted stream holds a lease: at least every T_{check} it compares every authority-bearing revision recorded in the disclosure plan (identity and delegation revisions, resource generation, metadata revision, access and label revisions of the parent and of each selected unit, context mark revision, policy epoch) and the decision's `valid_until` with their current values, without re-evaluating policy. If a comparison fails, or a check cannot complete within $2T_{check}$ because of overload or dependency failure, the connection is aborted so the client observes an error and not a normal end of stream; a check is never skipped under load. Cancellation by the client ends the checks. T_{check} has an initial target of 5 s and is qualified under T-22. Bytes already emitted are not recalled. (R1-C · T-13)

3.2 Publication protocol

PostgreSQL controls logical namespace and visibility. Public bucket names and object keys are never used as filesystem paths.

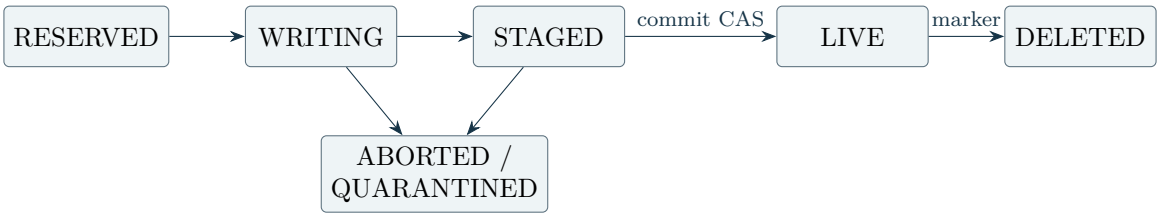


Figure 2: Catalog publication is the visibility boundary. Orphaned physical blobs never become readable merely because they exist. A DELETED version is physically retired only after the retention and reference checks of PUB-005.

[PUB-001] PostgreSQL controls logical namespace and visibility. Public bucket names and object keys **MUST NOT** be used directly as filesystem paths. (R1-C · T-08)

[PUB-002] Staging proceeds as follows. (1) Create an upload intent with tenant, actor, subject, delegation, stable object ID, expected object generation, expected current version, metadata revision and idempotency key. (2) Authorize create or new-version, and assign protected metadata on the server. (3) Stream authenticated input into an unpublished blob. (4) Validate body digest and length and finalize the blob through the backend contract, recording durable receipts, immutable identity and checksums; the version remains STAGED. (R1-C · T-08)

[PUB-003] Commit obtains a fresh commit decision and then, in one transaction, compares generation, current version, metadata, identity and delegation revisions, policy epoch, decision expiry and conditional-write preconditions; atomically publishes LIVE, updates the current pointer, increments the generation, consumes the intent and appends audit and outbox records. Only then is success acknowledged. (R1-C · T-08)

[PUB-004] A blob that is not referenced by a committed catalog version is never readable. A failed

commit leaves an orphan that is eligible for later reconciliation, not a visible object. (R1-C · T-08)

[PUB-005] A key deletion first publishes a logical delete marker and advances the generation; older versions remain subject to explicit historical rights. Permanent version deletion checks retention and version-delete permission, durably withdraws the version, updates any current pointer according to documented version semantics, and schedules physical deletion. Bytes are not removed before the logical transition is committed. (R1-C · T-08)

[PUB-006] Garbage collection uses catalog references, intents, tombstones and a grace period; file age is not evidence of abandonment. Reconciliation is idempotent after crashes before or after blob finalization and catalog commit. A restore recovers catalog and blobs to a mutually consistent point or quarantines unmatched entries. (R1-C · T-08)

[PUB-007] `If-Match` and `If-None-Match` apply to the committed logical version and its documented ETag, not to a staging file. Conflicting commits serialize with row locks or compare-and-swap and a fixed lock order, and two concurrent overwrites **MUST NOT** both publish against the same obsolete expected generation. An ETag is an opaque identifier and is not promised to be an MD5; plaintext and ciphertext checksums are stored separately. (R1-C · T-08)

[PUB-008] Version state moves only along RESERVED, WRITING, STAGED, LIVE and DELETED, with WRITING or STAGED able to move to ABORTED or QUARANTINED, as in Figure 2. (R1-C · T-08)

[PUB-009] The first release does not claim regulatory WORM or Object Lock compliance. (R1-C · REV)

3.3 Object backend contract

```
// Logical contract. All requests carry verified workload bindings.
type ObjectBackend interface {
    Begin(ctx context.Context, req BeginRequest) (StagedWriter, error)
    Finalize(ctx context.Context, req FinalizeRequest) (DurableReceipt, error)
    Open(ctx context.Context, req ReadRequest) (BlobReader, error)
    Remove(ctx context.Context, req RetireRequest) (RemovalReceipt, error)
}
```

[BKD-001] `Begin` binds operation, tenant, blob and placement. `Finalize` checks exact length and digest and returns a durable receipt only after the required file and directory flushes. Repeating the same identity and content is idempotent, and the same blob ID with different bytes rejects. A timeout is an unknown result reconciled by operation identity, not an assumed failed write. `Open` exposes exact immutable bytes, and `Remove` accepts only a current catalog-authorized retirement plan. The caller owns stream close, cancellation and logical publication. (R1-C · T-08, T-15)

[BKD-002][NEW] R1 runs on our own single-node blob store on local disk behind this contract (decision D-02). It writes each blob to an exclusively created temporary file, checks length and digest, flushes the file and its directory, and commits by rename before the catalog publishes the version. Durability and availability claims are those of one node plus backups and **MUST NOT** exceed them. Host disk encryption is not described as the per-unit cryptographic separation of R2. (R1-C · T-08)

[BKD-003] The local backend uses service-owned directories, restrictive permissions, descriptor-relative safe file operations and symlink and traversal protection. Finalized blobs are immutable, and file and required directory metadata are flushed before a catalog transaction publishes a LIVE reference. Storage clients have no direct filesystem or backend credentials. (R1-C · T-08)

[BKD-004] A different backend implements the same interface and needs its own consistency and durability tests. It does not duplicate user-facing policy combination in a second storage engine.

(R1-C · T-08)

3.4 Documents and text units

[DOC-001] A document version has an immutable, authenticated manifest listing its constituent text units in canonical order with their object and version identities, digests and the extraction and normalization version. The manifest digest is bound into read and export plans. (R1-C · T-06, T-08)

[DOC-002] Document publication atomically validates that every referenced unit and version is committed and has the required durable receipts, records protective references and advances the document generation in the catalog transaction. Incomplete ingestion remains unpublished and reconcilable, and staged internal units are not exposed through raw storage APIs. Garbage collection and retirement honor manifest references with the same lock and compare-and-swap discipline. (R1-C · T-08)

[DOC-003] Content is split at authorization boundaries into stable units such as sections, paragraphs or independently protected text entries; each unit is an existing immutable logical object. Boundaries are fixed at ingestion and re-splitting creates a new document version. Protected access labels and their revisions are separate mutable records. (R1-C · T-06)

[DOC-004] The first ingestion profile accepts bounded UTF-8 text and Markdown with a recorded normalization scheme. PDF, DOCX and other formats need a separately qualified extraction adapter with sandboxing, size and complexity limits and source-to-text mapping, and the platform makes no promise of universal document parsing. Extraction and indexing workers have explicit workload permissions and **MUST NOT** silently send plaintext to an external converter or embedding provider. (R1-C · T-09)

[DOC-005] A text collection or conversation uses the same unit model through `text.entry`; no separate transcript authorization store exists. Manifests, text units, source references and policy metadata are tenant-bound catalog records, and unit IDs are not bearer permissions. (R1-C · T-06)

[DOC-006] A document-level permission to enumerate or read units does not grant every unit. Actor, subject and delegation checks apply at both the parent and the selected-unit boundaries. An original binary or download needs a separate full-document permission that is never inferred from a unit read, and raw, preview, thumbnail, version, manifest and key APIs preserve that distinction. Internal unit blobs are not exposed through a broadly authorized raw object endpoint. (R1-C · T-06)

[DOC-007] An explicit unit-ID request requires every requested disclosure to be allowed; otherwise it returns a generic denial without forbidden content. An explicit authorized-subset operation returns only permitted units with an opaque continuation and no placeholders or counts that disclose hidden units. Source positions, titles and citations require their own permitted metadata projection. (R1-C · T-06, T-09)

[DOC-008] Streaming-AEAD segments and HTTP byte ranges are not independent authorization units. If different spans need different rights they are split into separate units before publication. (R1-C · T-06)

3.5 Labels

Authorization needs a trusted classification for every unit. Neither client-supplied nor model-generated labels are authoritative.

[LAB-001][NEW] Every published unit carries a protected classification label from the tenant's label order. Publication of an unlabeled unit is rejected. (R1-C · T-09)

[LAB-002][NEW] A label comes from, in order of precedence: (1) explicit labeling by an authorized principal within their delegation ceiling and at or above the collection's floor; (2) the collection or bucket default; (3) a proposal from a trusted classifier workload, which is advisory until accepted under (1). Client-supplied and model-generated labels are never applied directly. (R1-C · T-09)

[LAB-003][NEW] A unit whose label cannot be determined takes its collection's most restrictive default, and every collection declares one. (R1-C · T-09)

[LAB-004][NEW] Lowering the stored label of a unit or derived output is an explicit `relabel` action with its own audit record and, by default, an approval. The effective label of a derived output cannot go below its lineage maximum unless a `relabel` allow for that output says so (DRV-007). Raising a label is an ordinary `label` action. (R1-C · T-09)

[LAB-005][NEW] A label change advances the access revision of the affected unit or output. Decisions, disclosure plans and stream leases bind that revision (DEC-006, CNT-008). A label change does not require re-encrypting payload. (R1-C · T-09)

3.6 Disclosure

Table 11: MCP operations and their disclosure boundaries

MCP operation	Required action model	Disclosure boundary	Phase
`documents.search`	Document discovery plus permitted unit and snippet discovery and read	Filter candidates before snippet text reaches the client, a remote reranker or a model; no raw global hit counts	R1-C
`documents.read`	Parent read-units plus per-unit read	Exact document, version, manifest and permitted-unit selection; no implicit original-file access	R1-C
`text.read`	Collection or entry and version-scoped read	Same underlying unit and context resolver; no unguarded text endpoint	R1-C
`ai.ask` (optional)	Per-unit use-in-model, registered model submit and destination disclosure	The proxy builds the final permitted context and calls only the approved provider and model	R1-C
`resources/list`, `resources/read`	Same discovery and read actions as tools	URI resolves through registry and catalog; no direct file, arbitrary URL or raw storage bypass	R1-C
`documents.export`	All checks of the remote-export mapping (Table 15)	Compatible-client ciphertext and key envelope, separate from plaintext reads	R2

[DSC-001] Disclosure has three shared steps. `ResolveDocumentUnits` binds the document version, manifest digest, selected units, source lineage and current access revisions. `AuthorizeDisclosure` uses the bounded batch evaluator for parent and unit actions, current context, recipient and destination, and returns an exact immutable disclosure plan; a missing or misbound result denies. The storage service or proxy then discloses only what the plan lists.

MCP handlers translate requests and responses and **MUST NOT** implement a second copy of resolver, authorizer or decryptor. (R1-C · T-09)

[**DSC-002**] Authorization occurs before unauthorized plaintext enters a prompt, tool result, remote reranker, embedding API, log, cache or model context. Post-generation redaction and instructions to the model are not substitutes. (R1-C · T-09)

[**DSC-003**] Disclosure and the actual model or export outcome are recorded with unit and version identifiers and hashes, not with plaintext prompts, keys or full documents. Provider calls reuse invocation idempotency, budget and UNKNOWN-outcome handling. (R1-C · T-09)

[**DSC-004**] The same final per-unit checks apply to listings, citations, previews, notifications, prompt templates and subscriptions. An unsupported surface is disabled rather than assumed to be guarded by an SDK. (R1-C · T-09)

[**DSC-005**] Query caches are bound to tenant, authority, version and epoch and **MUST NOT** reuse a result computed for a broader user for a narrower one. No raw totals or structural positions reveal denied units. (R1-C · T-09)

[**DSC-006**] A policy that requires a particular model destination uses controlled submission and **MUST NOT** also grant unrestricted plaintext delivery or decrypt-capable material to arbitrary clients. Exporting a usable key is equivalent to plaintext disclosure for this constraint, and only an explicitly trusted controlled recipient that matches the allowed destination may receive such a capability. (R1-C · T-09)

3.7 Search

[**SRH-001**] Search begins with tenant-scoped catalog metadata and PostgreSQL text-search candidates where plaintext indexing is authorized. A vector database or RAG framework is not a baseline dependency. (R1-C · T-09)

[**SRH-002**] A plaintext or embedding index is sensitive derived data inside the trusted processing boundary. It is encrypted at rest and in backups with the same custody as the catalog, and entries for deleted or retired units are removed within a configured bound. Client-envelope data cannot become server-searchable plaintext without an explicitly authorized decryption and indexing step. (R1-C · T-09)

[**SRH-003**] A remote search or embedding service is a disclosure destination and requires its own policy decision. (R1-C · T-09)

[**SRH-004**]NEW Result values, ordering, scores and totals returned to a caller depend only on candidates that caller is authorized to see. Page shape (short or empty pages) and latency can reflect scan work over hidden candidates; they are bounded by the scan budget and tenant limits, but the platform does not claim that pagination is noninterfering with respect to hidden candidates. (R1-C · T-09)

3.8 Derived outputs and declassification

Summaries, extracted text and model answers are new resources whose confidentiality depends on what they were derived from.

[**DRV-001**]NEW A derived output records lineage: all units disclosed to the generating operation. Model-provided citations do not narrow it. Lineage is bounded and checked for cycles. (R1-C · T-09, T-21)

[**DRV-002**]NEW An output is delivered to a recipient only if (a) the recipient is allowed `read` on every unit in its lineage, or (b) an explicit `declassify` decision allows delivery of that output, at that version, to that recipient. A use-in-model permission or a controlled-route permission never

implies either, and a `declassify` allow authorizes that disclosure only: it does not change the stored label (DRV-008). (R1-C · T-09)

[DRV-003][NEW] The intended recipient and the conservative lineage are bound to the disclosure plan before output starts. Final wording is not inspected to decide whether protected material leaked, and streamed output follows the same rule. (R1-C · T-09)

[DRV-004] In controlled model submission, a client-requested model name resolves to a registered model, endpoint and credential profile, and the caller cannot substitute a URL after approval. Changing a parameter or destination invalidates the prepared plan. The proxy builds only permitted context and calls only the approved provider and model. (R1-C · T-09)

[DRV-005][NEW] When the requesting subject is allowed `use-in-model` but not `read` on some lineage unit, the model result is held and delivered only on a `declassify` allow for that output and recipient; otherwise it is discarded and the request is denied. (R1-C · T-09)

[DRV-007][NEW] The effective label of a derived output is at least the highest effective label in its lineage. A collection default acts as a floor, so the effective label is the maximum of the collection default and the lineage maximum. Only an explicit `relabel` allow for that output can lower it, and the effective label is the one used when the output is reopened and when a context mark is updated. (R1-C · T-09, T-21)

[DRV-008][NEW] Disclosure to one recipient and relabeling are separate authorizations. A `declassify` allow is bound to output, version and recipient and records the label at which the delivered copy is treated for flow control; absent an explicit lower label, that is the lineage maximum. Lowering the stored label for everyone is a `relabel` action (LAB-004). (R1-C · T-09, T-21)

[DRV-006] The model and provider that receive plaintext belong to that disclosure's trust boundary. Provider retention and training behavior is a contractual or technical property outside the cryptographic guarantee, released text cannot be recalled, and the conservative lineage model is not proof that an output contains no sensitive inference. (R1-C · REV)

3.9 Sizing and scaling

[SIZ-001][NEW] Qualification publishes a sizing model with the quantities in Table 12, uses the reference documents named there, and reports measured values. In R2 the model also covers key wraps, unwraps and release batches. (R1-C · T-22)

[SIZ-002] The MCP adapter scales with the `tools` role and the content service with the `storage` role, using bounded unit fan-out, parallel reads, key-release batches and context-size limits. No new permission database, duplicate evaluator, distributed cache or custom model-routing framework is introduced for the AI proxy. (R1-C · T-22)

Table 12: Sizing quantities (formulas; values are measured, not assumed)

Quantity	Formula, with u units per document version and c search candidates	Release
Decision batches to read a whole document	$\lceil (u+1)/50 \rceil$ (one parent check plus u unit checks)	R1-C
Decision batches per search page	$\lceil c/50 \rceil$ before any prefilter (CNT-007)	R1-C
Blobs, catalog rows per ingest	u blobs, u unit rows, 1 manifest	R1-C

Quantity	Formula, with u units per document version and c search candidates	Release
Key wraps at ingest, flat wrapping	u KMS wraps	R2
Key wraps at ingest, two-level (KEY-001)	1 KMS wrap plus u local wraps	R2
Durable receipts per publication	Backend-defined; 2 per unit with the custom store	R1-C, R3
Reference documents	$u = 10$, $u = 1{,}000$ and $u = 10{,}000$	R1-C

4 Key custody

Part 4 is required for release R2 and is not needed by R1. It keeps the cryptographic design of version 1.3, adds a two-level key hierarchy, and requires a signed decision for every key release. Encryption uses an established streaming authenticated-encryption library; the project implements custody, authorization, framing metadata and lifecycle, not new primitives.

4.1 Two custody profiles

Each immutable object version receives a fresh data key (keyset). A separately protected wrapping key protects it, and the wrapped keyset is stored in the catalog with an exact object and version binding. This is envelope encryption.

Property	Server-managed profile (default)	Client-envelope profile (optional)
Encryption location	The storage service encrypts and decrypts streams	The project SDK encrypts before upload and decrypts after download
Storage API result	Plaintext over authenticated TLS	Ordinary S3 clients receive ciphertext; the SDK restores plaintext semantics
Plaintext and key trust	Client, storage process, broker and KMS chain; an authorized proxy or model also sees disclosed plaintext but no raw storage keys	Client, broker and KMS chain; storage-only compromise does not obtain data keys
Backing blobs	Ciphertext with non-secret format metadata	Ciphertext with non-secret format metadata
Key release	Broker releases to the authorized storage workload, bound to the user or agent operation	Broker may release to an explicitly authorized client identity
Revocation limit	Blocks new admissions and key requests; an already-admitted stream may finish within its lease	Blocks new key releases; previously received keys remain usable offline

[CRY-001] A bucket or object profile is server-controlled and cannot silently downgrade to plaintext. S3 SSE-KMS header compatibility is not advertised until its semantics are implemented, and the platform's own encryption at rest is independent of those headers. The server-managed profile **MUST NOT** be described as excluding storage-service operators from plaintext access. (R2 · T-10)

4.2 Cipher suite and immutable binding

[CRY-002] Payload encryption uses a qualified streaming AEAD (initial candidate: Tink AES-256-GCM-HKDF with 1 MiB segments). The library handles nonce generation, key derivation, segment authentication and final-segment marking, and the project **MUST NOT** reproduce its chunked encryption. The module checksum and format tests are pinned. (R2 · T-10)

[CRY-003] Payload additional authenticated data (AAD) is the 60-byte value in Table 14. It binds tenant, object, crypto version, suite and format to both the payload and the wrapped key package. Mutable display names, classifications and wrapping-key rotation generation are not bound, and AAD is authenticated, not confidential, so no secrets or personal data are placed in it. (R2 · T-10)

Table 14: Payload AAD layout

Bytes	Size	Content
0–7	8	ASCII `FGAAD001`
8–23	16	Tenant UUID
24–39	16	Object UUID
40–55	16	Crypto-version UUID
56–57	2	Suite identifier, unsigned big-endian, from a versioned registry
58–59	2	Format version, unsigned big-endian

[CRY-004] The envelope row includes tenant ID, object ID, crypto-version ID, logical version ID, suite and format, wrapped keyset, provider and key reference, wrapping-key version, envelope generation, AAD bytes, ciphertext digest and length, and publication state. Object storage version, crypto version and envelope generation remain distinct. (R2 · T-10)

[CRY-005] Raw keysets are never persisted by platform services outside an approved protected key facility. An explicitly authorized client-envelope or export recipient can hold its released keyset with the stated custody and revocation limits, and client-supplied envelopes are not accepted as authority for arbitrary unwrap requests. (R2 · T-11)

4.3 Key hierarchy and broker

[KEY-001][NEW] Flat per-unit wrapping is the baseline: each unit keyset is wrapped directly by the tenant wrapping key in the KMS. If decision D-09 selects the two-level hierarchy on the strength of the sizing measurements, keys form three levels with separate formats. (1) A non-exportable tenant wrapping key in the KMS wraps each document-version key-encryption key (KEK) generated by the broker, with the binding `FGKEK001` followed by tenant UUID, document-version UUID, KEK UUID, suite and format (60 bytes, the layout of Table 14). (2) Each unit keyset is wrapped under that KEK by an AEAD whose associated data is the unit's own 60-byte payload AAD followed by the 16-byte KEK UUID, so a wrapped keyset cannot be moved to another unit, tenant or KEK. (3) The KEK exists in broker memory only for a bounded time-to-live, is evicted when its document version is retired or its authority is revoked, is never released, and authorizes nothing: every release still needs its own fresh signed decision. Independent per-unit release is preserved. (R2 · T-10, T-12, T-22)

[KEY-002] The broker independently validates workload identity and requires a signed decision (DEC-008) for the exact operation. It resolves key references from tenant configuration and never from caller-provided KMS URLs or key identifiers. Storage and tool credentials have no direct

KMS path that bypasses the broker, and tool-facing agents receive neither wrapping credentials nor raw data keys in the standard profile. (R2 · T-11)

[KEY-003] A pending upload may obtain its newly generated key for encryption but is not eligible for general read or decrypt-key release. Finalization proves storage and envelope binding; it does not prove that encrypted client data is harmless, correctly classified or decryptable, and AEAD validation occurs on decryption. (R2 · T-11)

[KEY-011] The broker operations behave as described in Table 16. (R2 · T-10, T-11)

[KEY-012][NEW] Remote export and ordinary-read key access are two distinct compound operations with the check sets of Table 15. The content service and the broker each verify the complete signed result against the table, and a decision that lacks any listed check is refused. (R2 · T-11)

Table 15: Compound operations for key release

Operation	Checks that must all be allowed and appear in the signed decision	Audience
Remote export (`documents.export`, `SealKeyRelease`)	`export` on the `document.version`; `export-key` on each selected `document.text_unit`; `receive-key` on the `crypto.recipient`; `release` on the `crypto.envelope` of each selected unit	The key broker
Ordinary-read key access (`ReleaseObjectKey`)	`read` on the unit or object, as decided for the read itself; `release` on its `crypto.envelope` to the storage workload	The key broker, for the storage workload

Table 16: Broker operations

Operation	Required behavior
`BeginEncryptedUpload`	Authenticate; authorize create or new-version and key creation; allocate intent and crypto version; generate a fresh keyset; wrap and persist it; release the plaintext keyset only to the allowed encryption endpoint over authenticated transport
`FinalizeEncryptedUpload`	Verify intent, object, version and profile, storage receipts, ciphertext length and digest, and current commit authorization; publish only through the catalog compare-and-swap protocol
`ReleaseObjectKey`	Ordinary-read key access: resolve a LIVE catalog version; verify that the signed decision covers the ordinary-read check set of Table 15 and every binding; unwrap the exact catalog envelope for the storage workload; record key-release audit
`SealKeyRelease`	Remote export: verify that the signed decision covers the complete remote-export check set of Table 15; validate recipient status; seal only the selected unit keysets to the registered recipient (RCP-005); record key-release audit
`RewrapEnvelope`	Authorize a maintenance operation; keep payload and keyset identity unchanged; use the approved version of the same named wrapping key; compare-and-swap the envelope generation and record rotation evidence

Operation	Required behavior
`RetireEnvelope`	Check retention, recovery and reference state; withdraw future unwrap authority; perform separately authorized cleanup without claiming that all key copies disappeared

4.4 KMS provider contract

```
type KeyProvider interface {
    Wrap(ctx context.Context, approvedKeyRef string,
        keyPackage, binding []byte) (Envelope, error)
    Unwrap(ctx context.Context, envelope Envelope,
        binding []byte) ([]byte, error)
    Rewrap(ctx context.Context, envelope Envelope,
        approvedTarget string, binding []byte) (Envelope, error)
}
```

[KEY-004] `Rewrap` rotates within the same key and rejects a different provider or key target. Cross-key migration is a separate privileged unwrap-and-wrap workflow with independent audit and rollback checks. (R2 · T-10, T-12)

[KEY-005] The first reference provider is OpenBao Transit as an external service, with non-exportable tenant wrapping keys, an AEAD cipher, derived keys and plaintext backup disabled. Wrapping is randomized, and retry idempotency belongs in the operation journal and the persisted wrap result, not in identical ciphertext. The broker has only the encrypt and decrypt privileges it needs, and rotation administration uses a separate identity. The derived-key context is exactly the base64 of the SHA-256 of the binding bytes, with no intervening serialization and no caller-chosen nonce. The provider's behavior for associated data and for rewrap is qualified before it is relied on, and deployment digests and licensing are reviewed independently. (R2 · T-10, T-12)

[KEY-006] The version-1 plaintext key package is: unsigned big-endian 16-bit format value 1; unsigned big-endian 16-bit binding length 60; exactly 60 binding bytes; unsigned big-endian 32-bit keyset length; then the library-serialized binary keyset. The parser rejects wrong lengths, trailing bytes, unknown versions and keysets above 64 KiB, and validates the suite and embedded binding before returning key material. (R2 · T-10)

[KEY-007] TLS and workload authentication protect all broker and KMS channels. KMS credentials are unavailable to ordinary data users and model runtimes. Key material, bearer tokens, full authorization headers and crypto package contents are redacted from logs, traces, crash dumps and support bundles. Memory cleanup is best effort and perfect zeroization of garbage-collected copies is not promised, and debugging and heap-profile access to key processes is restricted. (R2 · T-19)

4.5 Streaming, retries and copy

[CRY-006] Decryptors release only authenticated segments and report overall success only after valid stream termination. Corruption, segment reordering, duplication, wrong AAD, wrong key, truncation and forged final segments fail. Buffered bytes and concurrent crypto operations are bounded, and an arbitrarily large object is never loaded into memory to decrypt it. (R2 · T-10)

[CRY-007] Retrying an upload resends identical ciphertext for the same encrypted attempt. Restarting encryption allocates fresh cryptographic state, and a new immutable content version gets a fresh keyset and crypto version. Plaintext ranges are disabled initially. Generic S3 ranges

in client-envelope mode address ciphertext bytes, and range support for plaintext requires authenticated segment expansion and output trimming and is deferred. (R2 · T-10)

[CRY-008] Because AAD binds the logical object and crypto version, an ordinary byte copy to a new object is not a valid encrypted object. Copy decrypts through an authorized endpoint and re-encrypts with a fresh destination binding, or rejects. A later reference or alias feature requires explicit same-ciphertext semantics and **MUST NOT** silently weaken identity binding. (R2 · T-10)

4.6 Rotation, compromise, revocation and recovery

[KEY-008] The platform responds to each event as in Table 17. (R2 · T-12)

Table 17: Key events and required responses

Event	Required response
Routine wrapping-key rotation	Rewrap KEKs (or keysets, in flat mode) under the new approved version, compare-and-swap the envelope generation, test reads, and retain old key versions until all live and retained versions and backups are covered
Data-key compromise	Generate a new data key and ciphertext version; rewrapping the same compromised key is insufficient, and old ciphertext copies remain exposed to that key
Wrapping-key compromise	Assess exposed envelopes and historical backups; re-encrypt affected data with fresh data keys where necessary; rotation does not undo already recovered secrets
Grant or user revocation	Block new admissions and key releases at the effective epoch or revision; retained plaintext, client keys and admitted stream data cannot be recalled
Deletion or crypto-erasure	Track replicas, envelopes, snapshots, backups and legal retention; deleting one catalog row does not erase every recoverable key copy
KMS disaster	Restore tested key snapshots and the applicable unseal or recovery mechanism before expecting data recovery; blob and database backups without recoverable keys are insufficient

[KEY-009] Every key event records object and version, actor and subject, decision ID, bundle digest, envelope generation, provider key reference and outcome, without secrets. (R2 · T-11)

[KEY-010] Revocation blocks new releases, including retries and cached responses served after authority expires. It does not erase an exported data key, a captured ciphertext and key pair, plaintext, model context or a derived answer. Expiry in a release envelope is an admission and recipient-processing constraint, not a self-destructing key. Static-recipient HPKE does not promise forward secrecy after the recipient's private key is compromised. New content uses new keys and versions, and rewrapping a disclosed data key cannot revoke its old access. (R2 · T-13)

4.7 Remote recipients and HPKE delivery

The chosen baseline is online attribute and context authorization at the broker plus recipient-bound HPKE key delivery. The broker decides using current policy, and HPKE protects the selected key package for an authorized remote private-key holder. This provides attribute-based access through an online authority without an offline attribute-key infrastructure.

[RCP-001] Three key purposes are separate: trusted-issuer and request authentication; recipient public-key encryption; and KMS wrapping at rest. Signing keys used for workload or OAuth proof are not interchangeable with X25519 HPKE encryption keys. Each registry record binds tenant, principal or workload, immutable key fingerprint, purpose, suite, status, validity and revocation revision. A public key identifies a recipient but is not authorization. (R2 · T-03, T-11)

[RCP-002] Enrollment and rotation use an authenticated, authorized control operation that validates the public-key format and suite and binds the key to the intended recipient. Activation requires trusted provisioning or proof of the corresponding private key; a challenge is server-generated, short-lived, one-use, bound to the identity and fingerprint, and completed over the authenticated enrollment session. A request cannot supply an unregistered replacement key by reusing a familiar key ID. (R2 · T-03, T-11)

[RCP-003] Remote issuer keys come only from configured trusted discovery or JWKS endpoints with bounded refresh, redirect and egress and with issuer, audience, intended-use and algorithm checks. Caller-supplied `jku`, `x5u`, KMS URLs and key-server addresses are never dereferenced, and JWT key selection is not a general remote encryption-key directory. (R2 · T-03)

[RCP-004] Key delivery uses the qualified Tink hybrid-encryption implementation with HPKE suite `DHKEM_X25519_HKDF_SHA256_HKDF_SHA256_AES_256_GCM`. HPKE encrypts a small key package, not the bulk document, and its hybrid encryption alone does not authenticate the sender. (R2 · T-10, T-11)

[RCP-005] `SealKeyRelease` implements the remote-export mapping of Table 15. It resolves all values from authenticated state: recipient fingerprint and version, tenant, actor, subject and delegation, document, version and manifest digest, exact selected unit, object and crypto-version identities, operation and decision IDs, bundle epoch and digest, expiry, audience and a release nonce. It validates current recipient status and, for every selected unit, all checks of that mapping immediately before release, and the package contains only the selected unit keysets and their existing bindings. No whole-document key is returned for section-only rights. (R2 · T-11)

[RCP-006] A versioned Protobuf `ReleaseContextV1` has no maps or unknown fields, fixed UUID and byte representations, normalized timestamp units and a unique sorted unit-binding list. The broker serializes it once, transmits the exact bytes, and hashes and signs those same bytes; recipients retain and verify the received bytes rather than reserializing. Strict parsing rejects duplicate units, unsupported values, wrong bindings and oversized lists. The HPKE context information is a domain-separated SHA-256 of the exact context bytes. (R2 · T-10)

[RCP-007] The remote wire profile names the Tink hybrid-encryption framing and key encoding used by the qualified Go SDK, and passes an interoperability test with the actual remote client; Tink ciphertext is not assumed interchangeable with arbitrary raw-HPKE implementations. Ordinary MCP clients keep using the plaintext proxy path unless they support the profile. An export resource link points to the authenticated broker endpoint, not a publicly reusable storage or key URL. (R2 · T-10)

[RCP-008] If an envelope must be verified after it leaves the authenticated connection, the complete context and ciphertext are signed with a separately registered broker-signing key and verified with the qualified JOSE library, because anyone can encrypt to a public key. (R2 · T-11)

[RCP-009] Sender-constrained OAuth such as DPoP is an optional, explicitly negotiated profile that uses a maintained implementation and enforces its signature, method, URI, token-key binding, time, nonce and replay rules. It neither grants resource permission nor automatically binds the HTTP body or the separate HPKE recipient key, and base MCP support for it is not assumed. (R2 · T-03)

[RCP-010] ABE and proxy re-encryption are deferred. Table 18 records the decision, and a later proposal needs its own threat model and maintained-library qualification, not a parallel default implementation. (R2 · REV)

Table 18: Alternatives to the online broker

Approach	Fit for this platform	Decision
Online broker plus HPKE	Current attributes, destination and delegation checks, remote public-key recipients, existing key custody	Required remote-delivery profile
Plain MCP over TLS	Authorized text reaches a controlled client or approved model through the proxy	Required interoperable path
Pre-issued or offline grants	Permit future access without a fresh online release; retained HPKE packages already permit offline decryption of their own ciphertext	Deferred; release expiry is not erasure
Ciphertext-policy ABE	Offline decryption from issued attribute credentials and ciphertext policy	Deferred until an offline or untrusted-storage requirement justifies attribute authorities, key updates and policy-subset limits
Proxy re-encryption	Delegated transformation without revealing plaintext to a proxy	Deferred; adds a key-lifecycle subsystem the online broker does not need

ABE is not rejected as insecure; it solves a different operating model. Arbitrary dynamic policy rules, approvals and device changes do not automatically become offline cryptographic conditions, and attribute expiry does not invalidate an already retained key and ciphertext pair.

5 S3 compatibility and replicated storage

Part 5 is required only for release R3. R1 and R2 run over the backend contract of Section 3 and the publication protocol of Section 3.2. This part adds an S3 facade and, if decision D-02 selects it, the custom replicated blob store.

5.1 S3 compatibility profile v1

Implement protocols from public specifications with an explicit small surface. Version 1 supports header-signed SigV4 over TLS, bounded known-length single-chunk PUT, GET and HEAD, logical deletion, explicit historical version operations and ListObjectsV2 with filtered visibility. Bucket creation initially uses the control API. Multipart, server-side copy, presigned requests, browser POST, ACLs, public bucket policies, SigV2, SigV4a, streaming-signature modes and directory-bucket extensions are disabled until separately qualified.

[S3P-001] The server advertises "S3 API subset, compatibility profile v1". Unsupported requests receive documented protocol errors, the restrictions appear in client configuration examples and tests, and the platform does not promise that default S3 client configurations work. (R3 · T-07)

[S3P-002] The profile requires an explicit SHA-256 payload hash (including the empty-body hash) and rejects `UNSIGNED-PAYLOAD`, every `STREAMING-*` mode and unimplemented checksum or trailer

encodings. The body digest is verified before publication, because a valid header signature does not prove that all received bytes match the signed payload. (R3 · T-07)

[S3P-003] The SigV4 verifier is a new security-sensitive module built from the documented canonicalization and verification requirements; the AWS Go SDK signer is a client helper, not a server verifier. It validates credential scope, service and region, timestamp and skew, signed-header coverage, request canonicalization, signature (with constant-time comparison), credential state and payload integrity. S3 object paths are not normalized, because duplicated slashes and encoded bytes can distinguish keys. (R3 · T-07)

[S3P-004] S3 routing uses a dedicated raw handler that preserves exact signing and object-key semantics. Go's `ServeMux` cleans and redirects some paths and is not used for S3 routing, although management APIs may use it. (R3 · T-07)

[S3P-005][NEW] The compatibility matrix names each tested client (for example the AWS SDK for Go v2, boto3 and rclone) with the exact settings it needs, including request-checksum behavior, because recent AWS SDK releases enable default request checksums and chunked trailers that the profile rejects. Failures under default settings are documented. (R3 · T-07)

[S3P-006] Protocol tests cover duplicate headers and query parameters, percent encoding, literal plus, Unicode, host ambiguity, missing signed security headers, truncation, oversized bodies and time-bound credentials, and use independently authored cases and external client tests. (R3 · T-07)

[S3P-007] A read authenticates the request, resolves a committed logical object and version, loads trusted attributes, evaluates read or stat, and opens exactly that immutable blob, binding resource generation, metadata revision, profile and crypto version to the decision. (R3 · T-07)

[S3P-008] Conditional writes (`If-Match`, `If-None-Match`) map to the publication compare-and-swap of PUB-007, and the platform documents its precise error mapping to S3 behavior. (R3 · T-07)

[S3P-009] Multipart and copy enter the compatibility matrix only after dedicated tests. Multipart needs an upload binding, creator and delegation, protected intended metadata, part checks, resource limits and a fresh completion decision that uses the same publication compare-and-swap and rejects revoked authority; retries use intent idempotency. Copy checks the exact source version, source read including historical access, destination create or new-version, and explicit source-to-destination transfer restrictions. (R3 · T-07)

5.2 Replicated blob backend

The production topology for object bytes is three full replicas in distinct failure domains with two durable receipts required before publication. The system survives the loss of one replica or domain under the stated placement and durability assumptions and repairs back to three. Erasure coding is deferred.

[REP-001] The writer assigns an immutable blob ID and placement generation and sends identical ciphertext to three selected replicas. Each replica verifies length and digest, flushes bytes and directory state, and returns an authenticated receipt binding tenant, node identity and incarnation, failure domain, placement generation, blob and version ID, digest and length. Distinct approved failure domains are counted, not merely processes on one host. (R3 · T-15)

[REP-002] Publication requires two valid receipts and stores their placement bindings; it never publishes after only a local memory or disk acknowledgement. A background repair worker copies the identical ciphertext to complete the third replica and does not independently re-encrypt. Under-replicated objects are visible in health metrics and are repaired before planned removal. Receipts from obsolete placement generations or node incarnations cannot satisfy a new publication, and a failed catalog compare-and-swap leaves an orphan eligible for reconciliation. (R3 · T-15, T-17)

[REP-003] Read admission still uses the authoritative catalog and policy epoch. A read may use

one recorded healthy replica of the exact immutable blob, checks expected identity and authenticated segments, and fails over to another replica on corruption or failure. A full-object digest is confirmed only after the complete stream. Cached blob availability does not authorize a read when authoritative permission state is unavailable. (R3 · T-15)

[**REP-004**] Placement uses stable generated IDs, capacity-aware deterministic selection and failure-domain constraints. The chosen node set and placement generation are persisted and are not recomputed from current cluster membership. Rebalancing writes and verifies a replacement replica before updating placement and retiring the old one. (R3 · T-17)

[**REP-005**] Repair state is durable. In-memory queues are bounded wakeups and never the sole recovery record. Work is deduplicated by kind, tenant, blob version and target placement generation, and progress, retry time and terminal or unknown status are persisted. A paced scanner reconciles catalog placements, pending intents, tombstones and recorded replica availability and checks ciphertext integrity against stored digests without decryption keys, bounded by bytes, objects, time and tenant quota. Lost wakeups, restarts and interrupted scans do not erase required repair. (R3 · T-17)

[**REP-006**] Adding capacity affects eligible future placements and does not automatically move existing blobs. Planned evacuation is an explicit restartable administrative operation: stop new assignment; inventory referenced placements; copy and verify replacements; compare-and-swap the placement generation; verify the required replica and domain count; then retire old copies. Cancellation stops new work while preserving valid committed placements. (R3 · T-17)

[**REP-007**] Maintenance is a registered workload, not an identity-less bypass. An internal plan binds deployment, workload action, tenant and blob, permitted source and target nodes, node incarnation, job and fencing token, and placement generation. Repair may copy validated ciphertext for a recorded placement; it may not publish logical content, obtain plaintext keys or delete unrelated blobs. Bootstrap runs in a separate restricted state, never as an omitted-principal shortcut in ordinary traffic. (R3 · T-20)

[**REP-008**] Physical removal requires a current catalog-authorized retirement or tombstone plan and rechecks live references, applicable retention or hold state and current placement at execution. The same destructive-operation guard serves garbage collection, scans and administrative jobs, and job authority and stale fences are checked at the node boundary as well as in the scheduler. The user who requested maintenance is recorded separately from the executing workload. (R3 · T-17, T-20)

[**REP-009**] The write quorum is not lowered automatically. A partition that cannot obtain two durable domain receipts or a safe catalog commit rejects writes. (R3 · T-16)

[**REP-010**] Production internode RPC authenticates workload and node identity, validates typed scope, placement and fencing, bounds streams and messages, and reuses the same receipt and error types as the local backend. It does not duplicate user-facing policy combination in a second storage engine. (R3 · T-20)

[**REP-011**] Encrypted, access-separated backups of immutable objects are part of the restore test of AVL-005, and restore reconciles catalog references with blobs to a mutually consistent point or quarantines unmatched entries. (R3 · T-18)

6 Engineering and verification

Part 6 holds the rules about how the platform is built and shown to be correct: source provenance, code quality, dependencies, module structure, tests, performance and release gates. These rules are separate from product behavior and apply from the first commit (phase P0).

6.1 Source provenance and licensing

The project studies upstream systems to understand their mechanisms and writes its own code. Source is a research input and is not copied or translated.

[SRC-001] Implementation code and tests are independently authored. MinIO or Cerbos implementation material is not incorporated and no line-by-line translation is produced. Copying code, tests, comments, assets or expressive documentation is a separate incorporation decision with its own rights and license obligations; it cannot be relabeled as idea reuse. Renaming symbols, changing programming language or machine-assisted paraphrasing is not evidence of independent implementation. (P0 · REV)

[SRC-002] A mechanism evaluation matrix records, for each studied mechanism: source URL and revision; area reviewed; observed behavior or invariant; adopt, adapt or reject; rationale; new module or contract; acceptance tests; and provenance and license disposition. Reviewers distinguish an idea learned from a source from material actually incorporated. Private research snapshots are kept apart from release source and artifacts. (P0 · REV)

[SRC-003] General-purpose libraries and reference material are used after a targeted license and provenance review, with required notices retained and versions pinned. A permissive license can allow different terms for a larger work while preserving obligations for the included material, and a direct reuse proposal is assessed separately before incorporation. (P0 · REV)

[SRC-004][NEW] Before Phase 0 closes, a license record states for each studied or used upstream (including MinIO and Cerbos and each direct dependency) the license at the pinned revision and its consequence for the project's own code, and is reviewed by a qualified person. The record also states whether formal clean-room controls are required. No upstream code, binary or container is used, in the product or in tests (D-01, D-03). Version 1.3 did not state these licenses; the expected terms are listed in Appendix F as unverified. (P0 · REV)

6.2 Code quality and naming

[QUAL-001] Maintainable complexity is optimized, not source compression or delivery speed. Dead code, speculative abstractions, duplicate implementations, unused configuration and unnecessary tests are removed, while validation, error handling, bounded resources, durability checks and audit evidence are kept even when they cost code. No hard function-length, total-size or coverage-percentage quota overrides review judgment. (P0 · REV)

[QUAL-002] Every function has one coherent responsibility and names its actual operation using domain names such as `ResolvePrincipal`, `AuthorizeOperation`, `ReserveInvocation`, `PublishObjectVersion` and `ReleaseObjectKey`, not generic `Process`, `Manager` or `utils` abstractions. Distinct security concepts have distinct types (for example `ActorID` and `SubjectID`, `ObjectVersionID` and `CryptoVersionID`); request and response structs are typed; states and outcomes are explicit; and unknown enum values are validated at boundaries. (P0 · REV)

[QUAL-003] Trust assumptions, preconditions, side effects, ownership and lifetime, cancellation and failure outcomes are documented where they matter. `AuthorizeOperation` does not perform the protected write, and `Prepare` does not dispatch. A transactional function identifies its commit point and unknown-outcome behavior. `context.Context` is passed through I/O and blocking work, and the owner of each stream and buffer is defined. Errors carry stable classifications, preserve causes for internal diagnostics, and are redacted for clients. A panic is never recovered into an allow, and an irreversible effect is never retried without its idempotency contract. (P0 · REV)

[QUAL-004] Changes to authentication, policy semantics, delegation, key handling, publication, replication, fencing or external side effects receive independent technical review by someone qualified for the area. AI-assisted code receives the same review, and compilation is not evidence of

a correct security design. CI runs formatting, `go vet`, scoped static analysis, relevant unit and integration scenarios, the race detector on meaningful concurrent paths, vulnerability scanning and targeted fuzzing of parsers, signatures, envelopes and cursors. Unresolved findings are recorded and an unmitigated applicable security or correctness defect blocks release. (P0 · REV)

[QUAL-005] A rushed or hard-to-explain patch is not accepted to meet a date. Review may require simplification, a better-supported dependency, clearer names or more evidence, and a change to security behavior needs explicit requirements and migration notes, not an incidental refactor. (P0 · REV)

6.3 Libraries and dependencies

The libraries in Table 19 are selection candidates, not completed security reviews. Exact versions, checksums and toolchain compatibility are recorded at admission in `dependencies.lock.json`.

Table 19: Candidate libraries and what the project still owns

Concern	Candidate	Project responsibility
HTTP, TLS, context, logs	Go standard library	Listener, time and resource bounds, workload identity, redaction, role isolation
Policy expressions	CEL for Go	Environment, schema and error modes, platform obligations, immutable releases
Streaming and recipient encryption	Tink for Go	Tenant, unit and recipient binding, custody, recovery
Catalog and pooling	pgx; sqlc as a build-time generator	Explicit SQL transactions, tenant predicates, compare-and-swap and fencing
OIDC and JOSE	go-oidc; go-jose	Issuer, audience, rotation, expiry, delegation and freshness checks
Schema validation, YAML	A maintained JSON Schema library and Go YAML implementation	Pinned dialect, bounded decoding, duplicate and unknown-field rejection, no network schema loading
MCP transport	Official MCP Go SDK	Tool and resource handlers, document checks, registry trust, credential isolation
Internal RPC	Protobuf with qualified Go generators	One canonical contract, bounded messages, mTLS, deadlines
Key custody and HA	OpenBao Transit; PostgreSQL with an established HA coordinator	Provider constraints, verified quorum and fencing, backups, restore
S3 client conformance (R3)	AWS SDK for Go v2	Independent protocol verification; the client signer is not a server verifier

[LIB-001] A standard or third-party component must fit the required semantics. Go's `ServeMux` cleans and redirects some paths, so S3 signing and object-key semantics need a raw handler. Schema

validation after decoding cannot recover a duplicate JSON field already discarded. Library defaults are tested or configured explicitly, not assumed secure. (P0 · REV)

[**LIB-002**] Every direct dependency has an owner and a short record covering why it is needed; alternatives and the standard library; maintainers and support policy; release and security-response history; current advisories; test and conformance evidence; API stability; license and notices; transitive footprint; and the upgrade or replacement path. Popularity or frequent commits do not demonstrate maintenance quality, and a stable library is not rejected merely because it changes infrequently. Narrow maintained libraries are preferred over a large framework used for one helper. (P0 · REV)

[**LIB-003**] `go.mod`, `go.sum`, code generators and deployment images are pinned, an SBOM is produced and required notices are preserved. Vulnerability analysis runs in CI and periodically after release, and an applicable vulnerability requires a fix, removal, isolation or a documented time-bounded disposition. Updates are reviewed and tested and then deployed with the staged rollout and rollback discipline of product changes. Automatic unreviewed production upgrades are avoided, a necessary security update is not delayed to preserve a pin, and each dependency has security-notification routing and a tested replacement plan. (P0 · REV)

6.4 Module and role structure

```
fine-grained-access/
  go.mod  go.sum  dependencies.lock.json
  cmd/fga/                    # server/CLI dispatch for control, tools, storage, blob
  cmd/fga-decision/           # decision service entrypoint (own main package)
  cmd/fga-keys/               # key broker entrypoint (own main package)
  internal/
    identity/ delegation/ context/ policy/ bundle/ decision/
    enforcement/ approval/ reservation/ invocation/ audit/
    content/ documents/ labels/ search/ derived/
    catalog/ backend/ publication/
    tools/registry/ tools/executor/ tools/connectors/ mcp/
    s3/auth/ s3/handlers/ replicated/ placement/ maintenance/   # R3
    keybroker/ kms/ cryptoformat/ recipients/                   # R2
  pkg/decision/v1/  pkg/enforcement/
  sdk/go/  sdk/typescript/
  api/proto/  api/openapi/  api/jsonschema/
  migrations/  policy-examples/  conformance/  vectors/
  tests/integration/  tests/adversarial/  tests/chaos/
  deploy/  docs/research/  docs/decisions/  LICENSES/  NOTICE
```

The map identifies ownership and boundaries. It does not require an empty package for every future feature; cohesive code stays together and packages split when a real boundary warrants it.

[**MOD-001**] There is one Go module with a shared composition root. The initial product imports no MinIO or Cerbos source or private APIs, uses public DTO and client contracts at external boundaries and unexported implementation details internally, and builds the server roles `control`, `tools`, `storage` and (R3) `blob` from `cmd/fga`, the `decision` role from `cmd/fga-decision` and the `keys` role from `cmd/fga-keys` (MOD-004), plus `fga policy validate` and `fga admin`. (P0 · T-20)

[**MOD-002**] Each production process selects one role and initializes only its declared routes, dependencies, credentials and background work. Key custody, tool execution and blob serving run under separate identities, mounts, network policy and replica sets. An all-roles mode, if provided, is development-only and rejected by production configuration, and packaging does not grant tools direct KMS or backend access. (P0 · T-20)

[**MOD-003**] Production configuration is typed, versioned and validated before listeners become

ready, with deterministic file, environment and flag precedence. Unknown fields, insecure missing values and incompatible role capabilities are rejected. Decision and replica RPC require authenticated workload identity. Playground, reflection, debug and profiling endpoints, payload logging and administrative listeners are disabled unless explicitly needed, isolated and authorized. Reloadable settings are enumerated, and security-authority, key-provider and listener-role changes require a controlled restart or a separately reviewed activation path. (P0 · T-20)

[MOD-004][NEW] The `keys` role and the `decision` role are built from their own `main` packages that import only their declared internal packages. CI compares each role's transitive import graph with a checked-in allowlist, and adding a package to a role's graph needs review. This is a build boundary, not a second implementation of shared code. Roles roll out independently with compatible contracts. (P0 · T-20)

6.5 Keeping the code base small

[SMALL-002] Repetitive API clients and DTOs are generated from one canonical definition per contract, and SQL access uses reviewed SQL plus generated query bindings rather than hand-maintained parallel data-access models. Tool and resource schemas and wire contracts remain distinct domains. Generators are pinned, output is reproducible and generated files are not manually patched. (P0 · REV)

[SMALL-003] Start with one qualified backend and topology, one KMS provider and one real tool connector, and extend through narrow provider interfaces when requirements justify it. No plugin framework, custom cryptography, replacement database, custom consensus or broad protocol implementation is justified solely by potential future use. (P0 · REV)

[SMALL-004] Handwritten production code, generated code, tests, direct and transitive dependencies and duplicated security logic are tracked. New subsystems are reviewed for maintenance cost, and dead code and unused dependencies are removed. Maintainability and attack surface are measured as well as lines of code, because moving complexity into unchecked generated code or a large dependency is not automatically a reduction. (P0 · REV)

[SMALL-005][NEW] PostgreSQL provides approvals, reservations, invocation ownership, bounded repair queues and the audit outbox while its measured capacity suffices. Redis, Kafka or a workflow engine is added only for a concrete requirement with a reviewed failure model. SQL leases, idempotency and outbox delivery still need explicit implementation and tests, since a library does not supply the project's transaction semantics. (P0 · REV)

6.6 Testing

[TEST-001] Every retained test demonstrates a distinct required behavior, security invariant, failure mode or regression. Table 20 is an evidence checklist and not an instruction to create a test per row, function, requirement, adapter or layer. Near-duplicate tests are consolidated into table-driven scenarios, trivial accessors, generated code and library internals are not tested, and external behavior is preferred over mocks and snapshots. Test code meets the same naming, ownership and review standards as production code, and no coverage percentage or test count is a product objective. (P0 · REV)

[TEST-002] Similar failure cases are retained only when they protect distinct guarantees and the distinction is stated. Wrong-tenant access, an expired delegation, a stale commit generation, a lost quorum and ciphertext truncation exercise different boundaries; they may share a table-driven harness but none is proven by the success path alone. A regression test reproduces the defect with the smallest stable case, and fuzz targets focus on hostile input boundaries. (P0 · REV)

[TEST-003][NEW] Each test in Table 20 marked for vectors has machine-readable vectors in

`vectors/`. Implementations consume them as the conformance corpus. Adding, removing or changing a vector changes the specification version, and the specification check validates that every vector cites existing requirements that list its test. (P0 · LINT)

[TEST-004][NEW] Policy-evaluation vectors run against the engine in CI. A change to the engine's semantics changes the vectors and the specification version together. (R1-A · T-01)

6.6.1 Security and correctness matrix

Table 20: Acceptance tests

Test	Phase	Required evidence	Vectors
T-01	R1-A	One conformance corpus covers deny overrides across roles and scopes, platform ceilings (no business allow overrides a ceiling DENY), condition errors and default denial. Indexed and straightforward execution agree. Requests through different adapters produce identical canonical decisions.	yes
T-02	R1-A	Bundle validation rejects documents with unknown fields or undefined constructs, unregistered resource types or actions, and ceiling documents that contain an allow rule. Evaluation errors in business or ceiling rules deny the affected action. Hostile cost and cancellation limits stay bounded and request context stays isolated.	yes
T-03	R1-A	Forged actor, subject or tenant; expired or widened delegation; audience mismatch; untrusted key locations; recipient substitution; signed-decision bytes, audience, expiry and replay checks; concurrent single-use consumption across replicas; signing-key rotation overlap.	yes
T-04	R1-B	Changing a tool argument, destination, model or amount after approval fails. Unknown or conflicting obligations fail before output. The approver sees the canonical plan. Redirects and registry changes are rejected.	yes
T-05	R1-B	Concurrent approval consumption and budget reservation neither overspend nor duplicate dispatch. UNKNOWN outcomes keep their reservations, are reconciled by a replacement worker, and are closed only through the recorded resolution path; an ABANDONED closure keeps the charge, the consumed approval and the redispatch prohibition.	yes
T-06	R1-A, R1-C	One canonical-resource scenario. R1-A cases, against a test adapter: registry resolution, alias binding and revalidation, compound checks. R1-C cases: HTTP read, MCP resource read and tool read, denied-unit raw download, alias and version races and incomplete manifests.	yes
T-07	R3	SigV4 canonicalization, signature or body mismatch, path encoding, duplicate keys and headers, skew, unsupported payload modes and scope mismatch behave as specified; named S3 clients pass with their documented settings.	no

Test	Phase	Required evidence	Vectors
T-08	R1-C	The publication scenario covers concurrent compare-and-swap, staged units, interrupted manifest commit, conditional writes, deletion and garbage collection. No LIVE document references an unavailable or retired unit.	yes
T-09	R1-C	Shared disclosure scenarios cover listings, snippets, citations, model context, derived outputs and declassification, labels, and caches. No denied unit, name or metadata reaches a recipient, including verbatim-answer and export attempts.	yes
T-10	R2	One envelope and stream harness covers tenant, unit, manifest and recipient substitution, wrong exact context bytes, corrupted, reordered or truncated ciphertext, HPKE client interoperability and authenticated broker origin.	no
T-11	R2	Pending or denied units cannot release keys; unit-only rights cannot obtain raw-document keys; a release without a valid signed decision, or with a decision missing any check of Table 15, is refused (one table-driven negative case per independent prerequisite); broker, KMS or decision failure denies; ordinary agent credentials cannot unwrap.	yes
T-12	R2	Rotation preserves intended reads. A compromised-data-key test shows that rewrap alone cannot revoke old plaintext access. Restore includes usable retained keys.	no
T-13	R1-A, R1-C, R2	R1-A: admission and commit checkpoints honor expiry and every authority-bearing revision, including retries. R1-C: stream-lease checks, including a label-only change with unchanged content generation and a check that cannot complete in time. R2: recipient-bound releases. Retained plaintext, keys and already-emitted bytes keep the documented limits.	yes
T-14	R1-A	A failed bundle activation does not publish readiness. Concurrent requests and batches keep one bundle. Stale nodes and invalid production configuration reject. Readiness cannot substitute for fresh admission. Rollback is a new higher epoch.	yes
T-15	R1-A, R1-C, R3	R1-A: mandatory audit records are not lost under load or exporter failure. R1-C and R3: killing any one failure domain during upload, policy activation, key release and tool dispatch leaves acknowledged data, catalog and audit outcomes intact.	no
T-16	R1-C	Partitioning database coordination and storage nodes leaves primary fencing intact, no quorum downgrade, no stale-grant resurrection and no duplicate unsafe tool dispatch. A cancelled synchronous wait leaves a row visible but unreplicated: no dispatch or key release occurs until the durability barrier is proven.	yes
T-17	R3	One shared repair and movement scenario covers corruption, lost wakeups, interrupted scanning and changed membership. Durable work resumes and replica count recovers without premature deletion.	no
T-18	R1-C, R3	Restoring database, blob, key and policy backups to a documented point quarantines inconsistent references, and RPO, RTO and key recoverability are measured.	no

Test	Phase	Required evidence	Vectors
T-19	R1-A	Logs, traces, metrics and support bundles contain no secrets, raw keysets, inappropriate full arguments or cross-tenant identifiers.	no
T-20	R1-A	Production role and configuration expose only allowed routes and credentials and preserve independent scaling. The no-bypass harness covers workload authentication, stale maintenance fences, cross-role privilege boundaries and per-role import allowlists.	yes
T-21	R1-C	Context high-water mark and derived outputs: reading a restricted unit blocks a lower-clearance destination; marks only increase and a concurrent read and egress cannot dispatch on a stale mark; the mark survives a new connection under the same context key; unlabeled units count as highest; a derived output keeps its lineage maximum when reopened; a label-order change without a mapping raises the mark to highest; a declassify allow does not relabel.	yes
T-22	R1-A, R1-C, R2	Performance and sizing qualification of Section 6.7. R1-A: decision-path limits and batch sizes. R1-C: listing sparsity, the reference documents and the stream-lease interval. R2: key-wrap rates. Each states the limits and failure modes.	no

6.7 Performance qualification

[PERF-001][NEW] Benchmarks measure decisions separately from context lookups, storage payload transfer, crypto, KMS and tool latency. They report hardware, policy and context sizes, policy selectivity, tenant skew, concurrency, payload sizes, cache configuration and failure mode, and include 1-, 10- and 50-resource batches, candidate-index selectivity, concurrent activation and preparation memory, cancellation, sparse visible listings and large streaming objects. They report p50, p95 and p99 and queue time, not only averages. Capacity tests include one failed domain and ongoing repair. The initial engineering objectives are p95 under 10 ms for local pure policy evaluation and bounded memory per concurrent stream; end-to-end objectives are set after representative context, KMS and network measurements. These are targets for qualification, not existing capabilities, and any changed limit is recorded with why it preserves denial behavior and resource isolation. (R1-C · T-22)

6.8 Release gates

[GATE-001][NEW] A release is accepted only when its code meets the quality and naming rules; its dependencies pass maintained-library review; its test evidence is sufficient without redundant scenarios; its source-research matrix and provenance record support the selected license; its enabled adapters have complete operation coverage in the coverage matrix (OVR-001); its exit tests in Section 0.2 and their vectors pass; and its documentation repeats the non-guarantees. Build and source manifests, license notices, schemas, signed policy fixtures, runbooks, migration scripts, dashboards and rollback and restore instructions are delivered with it. (P0 · REV)

[GATE-002][NEW] A Phase 0 decision (Section 0.6) is closed and recorded before the work it gates starts. Closing one does not reopen the core design: independent implementation, explicit delegation ceilings, one shared policy evaluation, mandatory adapters, immutable stored content, brokered encryption and fail-closed semantics remain required. (P0 · REV)

[GATE-003][NEW] R1-A, R1-B and R1-C are subphases of one release. No subphase is deployed to

production by itself, and production use of R1 requires every R1 exit test to pass. (P0 · REV)

A Representative end-to-end scenarios

Each scenario names the requirements it exercises. They are illustrations; the vectors of Appendix B are the testable form.

A.1 An agent reads a document and submits an expense (R1-B, R1-C)

An authenticated agent acts for a user under a project-scoped delegation. The tool gateway prepares three checks in one batch: read the selected text unit, invoke the expense tool, and send to the registered expense service (RES-006). The document manifest and unit bindings are fixed before authorization (DOC-001).

```
{ "request_id": "r-81", "tenant_id": "t-acme", "adapter_id": "tools-gw-1",
  "operation_id": "op-9f3",
  "actor":      {"id": "agent-7", "type": "agent", "identity_revision": 12},
  "subject":    {"id": "user-42", "type": "user",  "identity_revision": 88},
  "delegation": {"id": "dlg-5", "revision": 3},
  "session":    {"id": "s-310", "binding_revision": 1, "context_key": "dlg-5",
                 "mark_revision": 4, "label_order_generation": 2},
  "checks": [
    {"check_id": "c1", "action": "read",
     "resource": {"type": "document.text_unit", "id": "u-1", "version": 4,
                  "generation": 17, "metadata_revision": 2}},
    {"check_id": "c2", "action": "invoke",
     "resource": {"type": "agent.tool", "id": "expense.submit", "version": 3,
                  "generation": 1, "metadata_revision": 1}},
    {"check_id": "c3", "action": "send",
     "resource": {"type": "network.destination", "id": "expenses.example",
                  "version": 1, "generation": 1, "metadata_revision": 1}}],
  "operation_digest": "sha256:...", "required_policy_epoch": 412 }
```

The amount is over the policy threshold, so the response is `NEEDS_APPROVAL`, with the approval summary rendered from the canonical plan (APR-005). After approval and a reservation, a new `ALLOW` whose context revisions include both is required (DEC-002). The executor dispatches with an idempotency key and a state precondition (AGT-006); a timeout becomes UNKNOWN and is reconciled (INV-003). Reading the document never grants submission authority.

A.2 Revocation during an upload (R1-C)

The uploader has a valid create decision and the blob is finalized. Before publication, project membership is revoked and its revision advances. The commit transaction compares the fresh decision's identity and delegation revisions and the object generation, rejects the old authority, and the catalog never publishes LIVE (PUB-003). The blob stays staged for reconciliation and later collection (PUB-006). A successful physical upload is not a successful authorized publication (vector V-061).

A.3 A remote AI reads only approved sections (R1-C; export in R2)

A remote analyst requests a report through MCP. Its actor and represented user may read the public overview and the financial appendix but not the personnel section. The proxy resolves the immutable manifest, checks parent and unit permissions and the requested model destination, and receives only approved plaintext units (DSC-001). Search snippets and citations follow the same projection (DOC-007). A direct original-file request is denied without full-document authority (CNT-003). In R2, a compatible encrypted export gives the analyst's registered recipient key an HPKE package containing only those units' keysets (RCP-005); revocation later blocks new releases but cannot recall keys or context already received (KEY-010).

A.4 A retrieved document tries to give instructions (R1-C)

A unit returned by search contains text telling the agent to register a new tool and widen its delegation. The text is data, not authority (MCP-004). The registry and delegation do not change, and any call the agent then makes is checked as an ordinary request (vector V-029). If the agent attempts to send content from that session to an external address, the context mark decides (next scenario).

A.5 The context high-water mark (R1-C)

During a session the agent reads a unit labeled *restricted*. The mark of its execution context becomes *restricted*; it is raised durably before the first byte is emitted and stays there (FLW-003, FLW-007). A later call to an email tool whose destination clearance is *internal* is denied, or goes to approval if a policy output asks for that (vectors V-110, V-111). If the agent reconnects with a new MCP session under the same delegation, the mark is inherited (FLW-008, vector V-114). A summary the agent derived from the unit keeps the restricted effective label even when it is saved in a public-default collection (DRV-007, vector V-119). A different delegation or context key starts at the lowest level, so content that reaches an agent by a route the platform does not mediate is not covered (FLW-005, N-02).

A.6 An external outcome is unknown (R1-B)

A payment call times out and the downstream service has no idempotency key support. The invocation becomes UNKNOWN and its reservation stays (INV-007). After the tool's maximum hold age, an operator other than the requester resolves it as failed, with evidence, and the reservation is released (INV-006, vector V-044); the requester's own attempt is denied (V-045).

A.7 Storage-node compromise (R2, R3)

An attacker reads disks on one blob node. It obtains ciphertext and non-secret storage metadata but no broker or KMS credentials, which protects payload confidentiality under the cryptographic assumptions (CRY-003). File sizes and placement metadata may still leak. If the attacker instead compromises the server-managed encryption process, that process handles plaintext and data keys and the guarantee does not apply (N-03). The client-envelope profile narrows that boundary at the cost of a dedicated SDK and client-held keys. A compromised gateway still cannot obtain keys that no signed decision covers (TRU-005).

A.8 One failure domain is lost (R1-C; blobs in R3)

Requests are routed away from failed gateways and decision nodes. A safely fenced database standby becomes primary when necessary (AVL-003). With the custom backend, each acknowledged blob has a surviving durable copy under the two-receipt rule and healthy nodes serve verified bytes and repair toward three copies (REP-002); with an external backend the backend's own guarantee applies (BKD-002). Tool invocations with uncertain external outcomes are reconciled rather than duplicated (INV-008), and backup recovery remains separate from this availability path (AVL-005).

B Test vectors

The vectors below are generated from `vectors/decisions.json`. Each gives a situation and the expected outcome, and cites the requirements it exercises. Policy-evaluation vectors use the format of Section 1.4.2 and run against our own engine (TEST-004). Outcome names beyond the five decision outcomes describe enforcement or storage results (for example `NOT_EXECUTED`, `UNKNOWN_RETAINED`, `FILTERED`).

ID	Given	Expected	Requirements
T-01 (R1-A)			
V-001	Principal has role reviewer, same tenant, and the unit's project is in the principal's projects; Unit state is LIVE; The example policy of Section 1.4.2 is active	`ALLOW`	POL-002, POL-003
V-002	Same principal and unit; Check made through the HTTP adapter and through MCP resources/read *Business decision and resource-bindings digest are identical (G-02).*	`ALLOW`, `ALLOW`	RES-003
V-003	Role reviewer requests action export on a document_unit; No rule grants export *Unresolved actions ultimately deny.*	`DENY`	POL-003
V-004	One role path has a matching DENY rule and a matching ALLOW rule for the same action	`DENY`	POL-002
V-005	Principal holds roles A and B; Role A's rule denies read; Role B's rule allows read *Deny overrides allow across roles (Section 1.4.2).*	`DENY`	POL-002
V-006	A business rule allows read for this principal; The ceiling set denies read when state is not LIVE; Unit state is DRAFT *No business allow overrides a ceiling.*	`DENY`	POL-007

ID	Given	Expected	Requirements
V-007	Batch of three checks; Second check is denied by policy *Results keep request order and the deny does not affect the others.*	`ALLOW`, `DENY`, `ALLOW`	DEC-004
V-008	A business rule denies read when principal.id is this principal; Another business rule allows read for the principal's role *Version 1 has no principal policies; a per-principal rule is a condition, and deny overrides allow.*	`DENY`	POL-003
T-02 (R1-A)			
V-010	A policy document has an unknown field, for example a lenient-mode setting	`REJECT_BUNDLE`	POL-002
V-011	A rule names an action that is not registered for its resource type	`REJECT_BUNDLE`	POL-002
V-012	A DENY rule's condition reads resource.attributes.classification, which is absent; A broader ALLOW rule matches the action *Strict evaluation: an erroring rule denies the affected action.*	`DENY`	POL-005, POL-006
V-013	A tenant-scope resource policy allows read of a DRAFT unit; The ceiling set denies read when state is not LIVE	`DENY`	POL-007
V-014	A policy document's format field is not fga.policy/v1 *Never silently ignored.*	`REJECT_BUNDLE`	POL-002
V-015	A condition exceeds the expression cost bound *A resource-limit failure is a platform failure, never an allow.*	`ERROR`	POL-009
V-016	A condition iterates over a ten-million-element list supplied in the request	`ERROR`	POL-009
V-017	A ceiling document contains an allow rule	`REJECT_BUNDLE`	POL-007
V-018	A ceiling rule's condition reads an attribute that is absent *A ceiling evaluation error denies the check.*	`DENY`	POL-007
T-03 (R1-A)			
V-020	Request body claims role admin; Roles resolved from authoritative state do not permit the action	`DENY`	ID-002

ID	Given	Expected	Requirements
V-021	Delegation expiry has passed *Platform reason DELEGATION_EXPIRED; business effects are NOT_EVALUATED.*	`DENY`	DLG-001
V-022	A child delegation lists an action that its parent does not	`DENY`	DLG-002
V-023	Delegation audience is executor A; Presented by executor B	`DENY`	DLG-001
V-024	Subject is permitted to invoke the tool; The agent actor lacks execution capability for the tool *Effective authority is the intersection.*	`DENY`	DLG-004
V-025	A signed decision whose audience is the storage role is presented to the keys role	`DENY`	TRU-004
V-026	A single-use signed decision for a mutation is presented a second time	`DENY`	DEC-008
V-027	The request carries an actor identity revision older than the current revision	`DENY`	ID-004
V-028	The MCP adapter calls the content service with its workload identity and caller-asserted claims only; No signed decision or verifiable delegation reference	`DENY`	TRU-003
V-029	A retrieved document says to register a new tool and widen the delegation; The agent only reads it *The read is allowed if otherwise permitted. The text causes no change to the registry or the delegation.*	`ALLOW`	MCP-004
V-120	After reading that document, the agent asks to register a new tool *An unauthorized follow-up is denied as an ordinary request.*	`DENY`	MCP-004
V-121	Two replicas present the same single-use signed decision at the same time	`EXECUTED_ONCE`, `DENY`	DEC-011
T-04 (R1-B)			
V-030	Expense of 500.00 USD was approved; The amount is changed to 5000.00 USD before dispatch *New operation digest, so the old approval no longer applies.*	`NEEDS_APPROVAL`	APR-001, AGT-004
V-031	The decision lists a required obligation type that the executor does not implement	`NOT_EXECUTED`	ENF-001

ID	Given	Expected	Requirements
V-032	The connector receives a redirect to a host that is not the registered destination	`NOT_EXECUTED`	AGT-005
V-033	The agent's description says Pay vendor X 50.00; The canonical plan says 5000.00 to vendor Y *The approver sees the canonical plan; the agent text is shown only as an unverified note.*	`RENDER_CANONICAL`	APR-005
V-034	The tool's schema digest differs from its registry record	`NOT_EXECUTED`	AGT-001
T-05 (R1-B)			
V-040	Two executions try to consume the same single-use approval at the same time	`EXECUTED_ONCE`, `DENY`	APR-002, APR-003
V-041	Remaining budget is 1000; Two concurrent reservations of 600 each	`RESERVED`, `DENY`	APR-004
V-042	Dispatch times out; The downstream service offers no idempotency or reconciliation *No redispatch; the reservation stays.*	`UNKNOWN_RETAINED`	INV-005, INV-007
V-043	A worker's lease expired while the invocation was DISPATCHING; A replacement worker picks it up	`RECONCILE`	INV-003
V-044	An UNKNOWN financial invocation is older than the tool's maximum hold age; A principal other than the requester records a failure verified by evidence	`RESERVATION_RELEASED`	INV-006
V-045	The original requester tries to resolve their own UNKNOWN financial invocation	`DENY`	INV-006
V-046	An operator marks an UNKNOWN financial invocation ABANDONED without verified evidence of failure *The reservation stays charged and the approval stays consumed; redispatch is denied.*	`RESERVATION_RETAINED`, `DENY`	INV-009
T-06 (R1-A, R1-C)			
V-035	Tool invoke is allowed; Source read is allowed; Send to the destination is denied *One denied check denies the compound operation.*	`DENY`	RES-006

ID	Given	Expected	Requirements
V-050	Same principal and unit; HTTP GET, MCP resources/read and the MCP read tool *S3 GET joins this set in R3.*	`ALLOW`, `ALLOW`, `ALLOW`	RES-002, CNT-001
V-051	An alias resolved to version 7; Version 8 is published before admission *Generation mismatch; the caller retries and resolves again.*	`DENY`	RES-004
V-052	Whole-document download is requested; The manifest has an unclassified attachment	`DENY`	CNT-003
V-053	The parent document allows read-units; The personnel unit is denied; A whole-document path is requested	`DENY`	CNT-003, DOC-006
V-054	An explicit unit-ID request mixes permitted and denied units *Generic denial; no content and no hint of which unit was denied.*	`DENY`	DOC-007
V-055	The same selection is sent as an authorized-subset request *Permitted units only, no placeholders and no counts of hidden units.*	`FILTERED`	DOC-007
V-056	An object key is renamed; The same principal is still allowed on the resource *Permission identity does not change with the alias.*	`ALLOW`	RES-005
T-08 (R1-C)			
V-060	Two writers stage versions against the same expected generation	`COMMITTED`, `REJECTED`	PUB-003, PUB-007
V-061	The uploader's project membership is revoked between staging and commit *The blob stays staged for reconciliation; the object is not LIVE.*	`REJECTED`	PUB-003
V-062	The process crashes after blob finalization and before catalog commit *Collected only after the grace period.*	`NOT_VISIBLE`	PUB-004, PUB-006
V-063	Manifest commit is interrupted after 3 of 5 units are committed	`NOT_VISIBLE`	DOC-002
V-064	Garbage collection finds an old blob that a document manifest still references *The deletion does not run.*	`NOT_EXECUTED`	PUB-006, DOC-002
T-09 (R1-C)			

ID	Given	Expected	Requirements
V-070	A search has 20 candidate units; 5 are denied to the caller *No total count; values and ordering depend only on the 15 permitted units. Page shape is not claimed to be noninterfering (SRH-004).*	`FILTERED`	SRH-004, DSC-005
V-071	An answer's citation metadata contains the title of a denied unit	`DENY`	DOC-007, DSC-004
V-072	A summary is derived from unit A (readable) and unit B (not readable by the recipient); No declassify decision exists	`DENY`	DRV-002
V-073	Same summary; A declassify decision allows delivery of this output to this recipient	`ALLOW`	DRV-002
V-074	Policy requires the controlled model route; The client requests raw text export	`DENY`	DSC-006
V-075	use-in-model is allowed on unit B; read is denied on unit B; The model produces a result *Delivered only on a declassify allow; otherwise discarded.*	`HELD`	DRV-005
V-076	A query result was cached for a user with broader rights; A user with narrower rights repeats the query	`NOT_REUSED`	DSC-005
V-077	Publication of a unit that has no classification label	`REJECTED`	LAB-001
V-078	A classifier workload proposes a lower label for a stored unit *Advisory until accepted by an authorized principal.*	`NOT_APPLIED`	LAB-002
V-079	HEAD on an object that exists but is hidden from the caller; HEAD on an object that does not exist *Both return the same response.*	`DENY`	CNT-004
V-080	A listing continuation token issued to principal P is replayed by principal Q	`DENY`	CNT-005
T-11 (R2)			
V-132	Remote export of selected units; Parent export on the document is denied; all other checks are allowed	`DENY`	KEY-012, RCP-005
V-133	Remote export of selected units; Export-key on a selected unit is denied; all other checks are allowed	`DENY`	KEY-012, RCP-005
V-134	Remote export of selected units; Receive-key on the recipient is denied; all other checks are allowed	`DENY`	KEY-012, RCP-005

ID	Given	Expected	Requirements
V-135	Remote export of selected units; Release on a selected unit's envelope is denied; all other checks are allowed	`DENY`	KEY-012, RCP-005
V-136	A validly signed decision for remote export lacks the receive-key check; It is presented to the broker *The broker verifies the complete check set, not only the signature.*	`DENY`	TRU-005, KEY-012
T-13 (R1-A, R1-C, R2)			
V-085	A delegation is revoked while a 200 MB stream is in progress *Aborted at the next lease check, within T_check; bytes already sent are not recalled (N-01).*	`ABORT_STREAM`	CNT-008, DLG-005
V-086	A decision is presented after its valid_until	`DENY`	DEC-005
V-087	The resource generation advances between decision and execution	`DENY`	DEC-006
V-088	A key-release request is retried after the grant was revoked *R2.*	`DENY`	KEY-010
V-122	A signed decision has a valid signature; The delegation revision has advanced since it was issued *A signature does not replace the revision check.*	`DENY`	DEC-006
V-129	A 200 MB stream is in progress; A unit's label is raised; the content generation is unchanged	`ABORT_STREAM`	CNT-008
V-130	A lease check cannot complete within twice T_check because the catalog is overloaded	`ABORT_STREAM`	CNT-008
T-14 (R1-A)			
V-090	A candidate bundle fails compilation on an instance *The prior snapshot stays active and readiness is unchanged.*	`REJECT_ACTIVATION`	REL-001
V-091	A node holds a bundle older than the tenant's required epoch	`DENY`	REL-003
V-092	A batch of 50 checks spans a bundle activation	`SAME_DIGEST`	REL-002
V-093	An older bundle is re-published as a rollback	`EPOCH_INCREASES`	REL-004
T-16 (R1-C)			

ID	Given	Expected	Requirements
V-123	A client cancels while the primary waits for synchronous replication; The transaction is visible on the primary; The standby flush position is behind the commit position *A lookup by operation ID is not evidence of durability; the outcome stays unknown.*	`NOT_EXECUTED`	AVL-004
V-124	Same transaction; A later synchronous commit is acknowledged, or the standby flush position reaches the commit position, on the fenced timeline	`EXECUTED_ONCE`	AVL-004
T-20 (R1-A)			
V-100	An ordinary API request asks to add a trusted issuer	`DENY`	ADM-002
V-101	The keys role's import graph contains the S3 handler package	`REJECT_BUILD`	MOD-004
V-137	The decision role's import graph contains the MCP SDK package	`REJECT_BUILD`	MOD-004
T-21 (R1-C)			
V-110	The session has read a unit labeled restricted; It calls an email tool whose destination clearance is internal	`DENY`	FLW-003
V-111	Same session and call; A policy output asks for approval for this case	`NEEDS_APPROVAL`	FLW-003
V-112	The session has read only public units; Same email call; all other checks pass	`ALLOW`	FLW-003
V-113	The session reads a unit with no label; It then calls the internal-clearance email tool *An unlabeled unit counts as the highest level.*	`DENY`	FLW-004
V-114	The context has read a restricted unit; The agent reconnects with a new MCP session under the same delegation; It calls the internal-clearance email tool *The mark is inherited by context key and is not reset by a connection.*	`DENY`	FLW-008
V-115	A session identifier bound to actor A is presented by actor B	`DENY`	MCP-007
V-116	A different delegation, and so a different context key, starts work *Content that reaches the agent by an unmediated route is outside the guarantee (N-02).*	`ALLOW`	FLW-008

ID	Given	Expected	Requirements
V-117	An internal-destination call is allowed at mark revision 5; A restricted read raises the mark to revision 6 before dispatch *Dispatch rechecks the mark revision; a new decision is required.*	`NOT_EXECUTED`	FLW-007
V-118	Two concurrent reads disclose a restricted unit and a confidential unit *Neither update can overwrite the other.*	`MARK_RAISED_TO_MAX`	FLW-007
V-119	A summary of restricted units is saved in a collection whose default label is public; It is reopened in a new connection under the same delegation; The agent calls the internal-clearance email tool *The effective label stays at the lineage maximum.*	`DENY`	DRV-007
V-127	A mark was computed under label-order generation 3; Generation 4 activates with a different order and no mapping; The context calls the internal-clearance email tool *An older-generation mark is treated as the highest level.*	`DENY`	FLW-004
V-128	A declassify decision allows delivery of a restricted-lineage output to recipient R; The output is later reopened in another context that calls the internal-clearance email tool *Declassification did not relabel the stored output.*	`DENY`	DRV-008

C Traceability

Generated by `tools/check_spec.py` from the requirement tags in the text. Every requirement lists a phase and at least one verification; every test is cited by at least one requirement.

Requirements per phase

Phase	Requirements	New
P0	29	10
R1-A	70	9
R1-B	30	3
R1-C	79	22
R2	32	3

R3	20	1
Total	260	48

Requirements verified by each test

Test	Phase	Requirements	Vectors
T-01	R1-A	DAT-001, DEC-001, DEC-002, DEC-003, DEC-004, DEC-010, DLG-003, ENF-003, POL-001, POL-002, POL-003, POL-004, POL-007, POL-008, POL-010, POL-011, POL-012, POL-014, RES-001, RES-003, RES-007, TEST-004	8
T-02	R1-A	POL-002, POL-005, POL-006, POL-007, POL-009, POL-013	9
T-03	R1-A	AGT-007, DAT-002, DEC-007, DEC-008, DEC-011, DEC-012, DLG-001, DLG-002, DLG-003, DLG-004, DLG-005, DLG-006, DLG-007, ID-001, ID-002, ID-003, ID-004, ID-005, ID-006, ID-007, MCP-004, RCP-001, RCP-002, RCP-003, RCP-009, TRU-001, TRU-002, TRU-003, TRU-004, TRU-006	12
T-04	R1-B	AGT-001, AGT-002, AGT-003, AGT-004, AGT-005, AGT-006, APR-001, APR-005, APR-007, ENF-001, ENF-002, ENF-003, ENF-004, ENF-005, ENF-006, MCP-004	5
T-05	R1-B	APR-002, APR-003, APR-004, APR-006, DAT-005, INV-001, INV-002, INV-003, INV-004, INV-005, INV-006, INV-007, INV-008, INV-009	7
T-06	R1-A, R1-C	ADP-001, CNT-001, CNT-002, CNT-003, DAT-003, DOC-001, DOC-003, DOC-005, DOC-006, DOC-007, DOC-008, MCP-001, MCP-002, MCP-003, MCP-006, RES-001, RES-002, RES-003, RES-004, RES-005, RES-006, RES-007, RES-008	8
T-07	R3	S3P-001, S3P-002, S3P-003, S3P-004, S3P-005, S3P-006, S3P-007, S3P-008, S3P-009	0
T-08	R1-C	BKD-001, BKD-002, BKD-003, BKD-004, DOC-001, DOC-002, PUB-001, PUB-002, PUB-003, PUB-004, PUB-005, PUB-006, PUB-007, PUB-008	5
T-09	R1-C	CNT-004, CNT-005, CNT-006, CNT-007, DOC-004, DOC-007, DRV-001, DRV-002, DRV-003, DRV-004, DRV-005, DRV-007, DRV-008, DSC-001, DSC-002, DSC-003, DSC-004, DSC-005, DSC-006, LAB-001, LAB-002, LAB-003, LAB-004, LAB-005, MCP-003, SRH-001, SRH-002, SRH-003, SRH-004	11
T-10	R2	CRY-001, CRY-002, CRY-003, CRY-004, CRY-006, CRY-007, CRY-008, KEY-001, KEY-004, KEY-005, KEY-006, KEY-011, MCP-005, RCP-004, RCP-006, RCP-007	0
T-11	R2	CRY-005, KEY-002, KEY-003, KEY-009, KEY-011, KEY-012, RCP-001, RCP-002, RCP-004, RCP-005, RCP-008, TRU-005	5
T-12	R2	KEY-001, KEY-004, KEY-005, KEY-008	0
T-13	R1-A, R1-C, R2	CNT-008, DAT-003, DAT-004, DEC-005, DEC-006, DLG-005, KEY-010, REL-003	7
T-14	R1-A	DAT-004, REL-001, REL-002, REL-003, REL-004, REL-005, REL-006	4

Test	Phase	Requirements	Vectors
T-15	R1-A, R1-C, R3	AUD-001, AUD-002, AVL-001, AVL-005, BKD-001, REP-001, REP-002, REP-003	0
T-16	R1-C	AVL-001, AVL-002, AVL-003, AVL-004, REP-009	2
T-17	R3	REP-002, REP-004, REP-005, REP-006, REP-008	0
T-18	R1-C, R3	AVL-005, REP-011	0
T-19	R1-A	AUD-003, KEY-007, REL-006	0
T-20	R1-A	ADM-001, ADM-002, AGT-008, MOD-001, MOD-002, MOD-003, MOD-004, REP-007, REP-008, REP-010, TRU-002	3
T-21	R1-C	DRV-001, DRV-007, DRV-008, FLW-001, FLW-002, FLW-003, FLW-004, FLW-005, FLW-007, FLW-008, MCP-007	12
T-22	R1-A, R1-C, R2	AVL-006, AVL-007, CNT-007, DEC-009, KEY-001, PERF-001, SIZ-001, SIZ-002	0

Requirements verified by review or by the specification check

ADM-003, AUD-004, DRV-006, FLW-006, GATE-001, GATE-002, GATE-003, LIB-001, LIB-002, LIB-003, OVR-001, OVR-002, OVR-003, PUB-009, QUAL-001, QUAL-002, QUAL-003, QUAL-004, QUAL-005, RCP-010, SMALL-002, SMALL-003, SMALL-004, SMALL-005, SRC-001, SRC-002, SRC-003, SRC-004, TEST-001, TEST-002, TEST-003.

D Review response (2.0 to 2.1)

An external review of version 2.0 raised seven prioritized findings, decisions on the author's choices and several contract clarifications. This table records how each was handled. Two sourced claims were checked before acting: Cerbos evaluates principal policies first and treats their explicit result as final for that action, and a cancelled synchronous wait leaves a transaction visible but not yet replicated. The reviewer received the PDF only, so `vectors/` and `tools/check_spec.py` were not examined; the checker is extended in this revision (GATE-003).

Item	Disposition	Where
F-01	Accepted. Root-scope rules are not a ceiling because a principal-policy result is final. Ceilings are now a separate deny-only set evaluated for every check by the same evaluator contract; the mandatory parental-consent rule is dropped and scope inheritance is declared business behavior.	POL-003, POL-007, POL-014, REL-001; vectors V-006, V-013, V-017, V-018
F-02	Accepted. Lookup by operation ID is replaced by a durability barrier proven before success, dispatch or key release.	AVL-004, INV-002; vectors V-123, V-124

Item	Disposition	Where
F-03	Accepted. Atomic durable mark raise before disclosure, mark revision and label-order generation bound into the decision, recheck at egress dispatch.	FLW-007, DEC-006, DEC-008; vectors V-117, V-118
F-04	Accepted. A derived output's effective label is at least its lineage maximum; declassify and relabel are separate.	DRV-002, DRV-007, DRV-008, LAB-004; vectors V-119, V-128
F-05	Accepted, by binding every authority-bearing revision directly.	CNT-008, LAB-005, DEC-006; vectors V-129, V-130
F-06	Accepted. One compound-operation table for remote export and one for ordinary-read key access; the broker verifies the complete signed result.	KEY-012, RCP-005, TRU-005; vectors V-132 to V-136
F-07	Accepted. R1-A, R1-B and R1-C are one release; shared tests carry per-phase cases; the checker rejects a requirement whose test first applies in a later phase. Tests T-06, T-13, T-15 and T-22 gained earlier cases, and INV-008 no longer cites T-16.	GATE-003; Table 20
Decision 1	Supported. The provider's durability and precondition behavior is qualified, and backend encryption is not presented as R2 separation.	BKD-002
Decision 2	Strict-only production profile kept and documented as a deliberate narrowing of the compatible tuning options.	POL-005
Decision 3	Supported. Signed bytes and audience defined; single-use consumption atomic and durable across replicas; signing-key lifecycle separate.	DEC-008, DEC-011, DEC-012
Decision 4a	Kept with a state contract. The mark is keyed by a trusted context lifetime, defaulting to the delegation, and survives reconnects; label order is immutable per generation with conservative handling of older marks. This revision chose the delegation default over limiting the claim to cooperating clients.	FLW-004, FLW-008; vectors V-114, V-116, V-127
Decision 4b, 4c	Mandatory lineage-derived classification; collection default is a floor only; recipient-specific declassification separated from global relabeling.	DRV-007, DRV-008, LAB-004

Item	Disposition	Where
Decision 4d	ABANDONED defined: charge kept, approval consumed, no redispatch, release only by verified failure or an explicit write-off. Hold age escalates and never infers failure.	INV-006, INV-009; vectors V-044, V-046
Decision 4e	Stream lease checks all revisions in the disclosure plan; fails closed under overload.	CNT-008
Decision 4f	The KEK hierarchy is conditional on D-09 and flat wrapping is the baseline. Formats are separate and the unit-wrap AAD now carries the unit's own binding. Convergent encryption is removed.	KEY-001, KEY-005
Decision 5	D-01 rewritten: evaluate the upstream engine first, native by deliberate choice; same corpus and epoch binding either way.	Section 0.6
Clarifications	Session reset and label order: FLW-004, FLW-008. Search privacy claim narrowed to returned values: SRH-004. Module tree aligned with the build boundary: MOD-001, MOD-004, vector V-137. Reading hostile text is no longer a denial: vectors V-029 and V-120.	as listed

E Mapping from version 1.3

Each of the 85 requirements of version 1.3 appears once. "Split" means the original bundled several obligations that are now separate requirements; "moved" means the content is unchanged but lives in another part; "kept" means one successor with the same obligation. Requirements marked new in the text have no version 1.3 predecessor, or strengthen one (for example POL-012, DEC-008).

1.3 ID	Result	2.0 requirements	Note
SRC-001	kept	SRC-001	Moved to Part 6.
SRC-002	kept	SRC-002	Moved to Part 6.
SRC-003	kept	SRC-003	Moved to Part 6; license record added as SRC-004.
QUAL-001	kept	QUAL-001	Moved to Part 6.
QUAL-002	kept	QUAL-002	Moved to Part 6.
QUAL-003	kept	QUAL-003	Moved to Part 6.
QUAL-004	kept	QUAL-004	Moved to Part 6.
QUAL-005	kept	QUAL-005	Moved to Part 6.

1.3 ID	Result	2.0 requirements	Note
LIB-001	split	LIB-001, S3P-004	Raw S3 routing requirement moved to the S3 profile.
LIB-002	kept	LIB-002	Moved to Part 6.
LIB-003	kept	LIB-003	Moved to Part 6.
MOD-001	kept	MOD-001	Moved to Part 6.
MOD-002	kept	MOD-002	Moved to Part 6; per-role import allowlist added as MOD-004.
MOD-003	kept	MOD-003	Moved to Part 6.
UNIFIED-001	split	CNT-001, CNT-002, TRU-006	Content service and the MCP principal rule separated.
SMALL-001	moved	ENF-003	Adapter rule moved to the enforcement part.
SMALL-002	kept	SMALL-002	Moved to Part 6.
SMALL-003	kept	SMALL-003	Moved to Part 6.
SMALL-004	kept	SMALL-004	Moved to Part 6.
ID-001	split	ID-001, ID-002, ID-003, ID-004, ID-005, ID-006, ID-007	Also absorbs the context-provenance prose of 1.3 Section 4.5 and the token-profile prose of Section 3.3.
ID-002	split	DLG-001, DLG-002, DLG-003, DLG-004, DLG-005, DLG-006, DLG-007	One obligation per requirement.
CANON-001	split	RES-001, RES-002, RES-003, RES-006	Compound-operation rule from Section 4.2 prose added.
CANON-002	split	RES-003, RES-004, RES-005, RES-007, RES-008	
CANON-003	moved	CNT-003	Content-specific; moved to Part 3.
COMPAT-001	split	POL-001, POL-002, POL-013	Evaluator contract (POL-001) makes the engine replaceable; YAML front end from Section 3.3 prose added.
COMPAT-002	split	POL-003, POL-004, DLG-003	
COMPAT-003	kept	POL-012	Strengthened with a differential reference oracle.
POL-001	split	POL-008, POL-009	
POL-002	split	POL-003, POL-007	Platform-ceiling rule added as POL-007.
POL-003	split	POL-011, APR-007, APR-001, APR-002, APR-003	Approval prose of Section 5.5 moved to enforcement.

1.3 ID	Result	2.0 requirements	Note
POL-004	kept	POL-010	
POL-005	split	POL-005, POL-006	Strict evaluation is now required in production.
AUTH-001	split	DEC-002, DEC-003, DEC-010	
AUTH-002	split	DEC-007, DEC-008, TRU-001, TRU-002	Signed decisions added.
AUTH-003	split	DEC-005, DEC-006, DEC-009	
AUTH-004	split	ENF-001, ENF-002, ENF-003, ENF-004	
AUTH-005	split	DEC-004, DEC-001	
DATA-001	split	DAT-001, DAT-002	
DATA-002	split	DAT-003, DAT-004	Cache rules of Section 7.2 included.
DATA-003	split	APR-004, APR-006, DAT-005, INV-007	
STO-001	split	BKD-003, PUB-001	
STO-002	split	S3P-001, S3P-002, S3P-009	Moved to Part 5 (R3).
STO-003	split	S3P-002, S3P-003, S3P-006	Moved to Part 5 (R3).
STO-004	split	S3P-007, CNT-004, CNT-008	Stream lease bound added.
STO-005	split	PUB-002, PUB-003, PUB-007, PUB-008, S3P-008	
STO-006	split	PUB-004, PUB-005, PUB-006, PUB-009	Deletion states added to the state machine.
STO-007	split	CNT-005, CNT-006	No-hidden-effects prose of Section 8.7 included.
STO-008	split	BKD-001, BKD-004, REP-010	
AGT-001	split	AGT-001, AGT-002, AGT-007, AGT-008	Section 9.3 prose included.
AGT-002	split	AGT-003, AGT-004, AGT-005, AGT-006	Invocation protocol of Section 9.2 included.
AGT-003	split	INV-001, INV-002, INV-003, INV-004, INV-005, INV-008	UNKNOWN resolution added as INV-006.
AGT-004	split	FLW-001, FLW-002, FLW-006	Session high-water mark added as FLW-003 to FLW-005.
ADP-001	split	ENF-005, ENF-006, ADP-001	
AI-001	split	MCP-004, DSC-006	
AI-002	split	DSC-001, DSC-002, DSC-003, DRV-004	Section 10.5 steps included.
DOC-001	split	DOC-001, DOC-002, DOC-003, DOC-004, DOC-005, DOC-008	Section 10.2 prose included.
DOC-002	split	DOC-006, DOC-007	
DOC-003	split	DSC-004, DSC-005, SRH-001, SRH-002, SRH-003, DRV-006	Derived-output rules rewritten as DRV-001 to DRV-005.
MCP-001	split	MCP-002, MCP-003, MCP-006	

1.3 ID	Result	2.0 requirements	Note
MCP-002	split	MCP-003, MCP-001, MCP-005, SIZ-002	Section 10.7 scaling prose included.
CRY-001	kept	CRY-001	
CRY-002	split	CRY-002, CRY-003, CRY-004, CRY-005	
CRY-003	split	CRY-006, CRY-007, CRY-008	
KEY-001	split	KEY-002, KEY-003, KEY-011, KEY-004, KEY-005, KEY-006	Sections 12.3 and 12.4 prose included.
KEY-002	kept	KEY-007	
KEY-003	split	KEY-009, AUD-004	
KEY-004	split	RCP-001, RCP-002, RCP-003	
KEY-005	split	RCP-004, RCP-005, RCP-006, RCP-007, RCP-008	
KEY-006	split	KEY-010, KEY-008, RCP-009, RCP-010	Rotation table of Section 12.6 included.
REL-001	split	REL-001, REL-002, REL-004	
REL-002	kept	REL-003	
OPS-001	moved	REL-006	
ADM-001	split	ADM-001, ADM-002, ADM-003	
AUD-001	kept	AUD-003	
AUD-002	split	AUD-001, AUD-002	
HA-001	split	REP-001, REP-002	Moved to Part 5 (R3).
HA-002	kept	REP-003	Moved to Part 5 (R3).
HA-003	split	AVL-002, AVL-003, AVL-004, REP-009	Control-plane part moved to Part 1.
HA-004	split	AVL-005, REP-011	
HA-005	split	REP-004, REP-005, REP-006	
MAINT-001	kept	REP-007	
MAINT-002	kept	REP-008	
SCALE-001	split	AVL-001, AVL-006, AVL-007	Topology table of Section 14.1 included.
TEST-001	kept	TEST-001	
TEST-002	kept	TEST-002	

New in version 2.0

APR-005, BKD-002, CNT-007, DEC-008, DEC-011, DEC-012, DRV-001, DRV-002, DRV-003, DRV-005, DRV-007, DRV-008, FLW-003, FLW-004, FLW-005, FLW-007, FLW-008, GATE-001, GATE-002, GATE-003, INV-006, INV-009, KEY-001, KEY-012, LAB-001, LAB-002, LAB-003, LAB-004, LAB-005, MCP-007, MOD-004, OVR-001, OVR-002, OVR-003, PERF-001, POL-012, POL-014, REL-005, S3P-005, SIZ-001, SMALL-005, SRC-004, SRH-004, TEST-003, TEST-004, TRU-003, TRU-004, TRU-005.

F Carried-over items not verified in this revision

Versions 2.0 and 2.1 restructure and rewrite version 1.3 and did not repeat the upstream research, apart from the two sourced claims named in Appendix D. The following items come from version 1.3 and have not been re-checked; each should be confirmed before the decision or requirement that depends on it is relied on.

- **Version pins and dates** in the reference list (the Go toolchain, CEL for Go, Tink for Go, OpenBao, PostgreSQL 18 documentation, and MCP protocol revision 2026-07-28). The specification itself pins nothing; pins are recorded at dependency admission (LIB-003).
- **Upstream commit hashes and source anchors** for MinIO and Cerbos. The functionality map that version 1.3 cites is now available as a private research snapshot (`fine-grained-access-research`, not part of the repository). It holds 96 MinIO and 69 Cerbos feature records, not the 97 and 71 that version 1.3 states. Its records describe behavior and link to pinned sources; they are used for ideas only (D-03).
- **Upstream license terms.** Version 1.3 does not state them. Expected here: AGPL-3.0 for MinIO and Apache-2.0 for Cerbos. A reviewer reports that the pinned Cerbos LICENSE is Apache-2.0; that report has not been verified here. The question is the subject of SRC-004 and decision D-01.
- **Third-party behavior that requirements rely on:** OpenBao Transit associated-data and rewrap behavior (KEY-005); the Tink hybrid-encryption suite name and framing (RCP-004, RCP-007); the default checksum behavior of current AWS SDK releases (S3P-005); and PostgreSQL's behavior when a synchronous wait is cancelled (confirmed against the Patroni documentation, AVL-004). No requirement relies on upstream policy-engine semantics any more.
- **Proposed numbers:** the decision limits of DEC-009, the lease check interval of CNT-008, the maximum hold age of INV-006, the replication counts and the five-minute recovery objective of AVL-005.

References

[1] MinIO mapped IAM and request enforcement. Source anchor from completed functionality mapping. https://github.com/minio/minio/blob/7aac2a2c5b7c882e68c1ce017d8256be2feea27f/cmd/iam.go#L2492

[2] Cerbos mapped effect combination. Source anchor from completed functionality mapping. https://github.com/cerbos/cerbos/blob/ae014ba2ea919771216bc666c4f654d18a1c1a52/internal/ruletable/check.go#L434

[3] Cerbos mapped schema/evaluation configuration. Source anchors from completed functionality mapping. https://github.com/cerbos/cerbos/blob/ae014ba2ea919771216bc666c4f654d18a1c1a52/internal/schema/conf.go#L10 and https://github.com/cerbos/cerbos/blob/ae014ba2ea919771216bc666c4f654d18a1c1a52/internal/evaluator/conf.go#L51

[4] Cerbos mapped server transport configuration. Source anchor from completed functionality mapping. https://github.com/cerbos/cerbos/blob/ae014ba2ea919771216bc666c4f654d18a1c1a52/internal/server/server.go#L264

[5] Cerbos mapped rule-table index and compilation. Source anchor from completed functionality mapping. https://github.com/cerbos/cerbos/blob/ae014ba2ea919771216bc666c4f654d18a1c1a52/internal/ruletable/index/index.go#L56

[6] Cerbos mapped batched API evaluation. Source anchor from completed functionality mapping. https://github.com/cerbos/cerbos/blob/ae014ba2ea919771216bc666c4f654d18a1c1a52/internal/svc/cerbos_svc.go#L258

[7] Cerbos mapped rule-table activation. Source anchor from completed functionality mapping. https://github.com/cerbos/cerbos/blob/ae014ba2ea919771216bc666c4f654d18a1c1a52/internal/ruletable/manager.go#L105

[8] MinIO mapped storage contracts and drive operations. Source anchor from completed functionality mapping. https://github.com/minio/minio/blob/7aac2a2c5b7c882e68c1ce017d8256be2feea27f/cmd/storage-interface.go#L29

[9] MinIO mapped recovery hints and scanning. Source anchors from completed functionality mapping. https://github.com/minio/minio/blob/7aac2a2c5b7c882e68c1ce017d8256be2feea27f/cmd/mrf.go#L78 and https://github.com/minio/minio/blob/7aac2a2c5b7c882e68c1ce017d8256be2feea27f/cmd/data-scanner.go#L155

[10] MinIO mapped pool evacuation mechanism. Source anchor from completed functionality mapping. https://github.com/minio/minio/blob/7aac2a2c5b7c882e68c1ce017d8256be2feea27f/cmd/erasure-server-pool-decom.go#L1473

[11] MinIO mapped readiness and quorum probes. Source anchor from completed functionality mapping. https://github.com/minio/minio/blob/7aac2a2c5b7c882e68c1ce017d8256be2feea27f/cmd/healthcheck-handler.go#L132

[12] MinIO mapped audit emission and target boundary. Source anchor from completed functionality mapping. https://github.com/minio/minio/blob/7aac2a2c5b7c882e68c1ce017d8256be2feea27f/internal/logger/audit.go#L63

[13] Model Context Protocol. *Resources*, protocol revision 2026-07-28. https://modelcontextprotocol.io/specification/2026-07-28/server/resources

[14] Google Tink. *Hybrid Encryption*. Recommended HPKE suite and separate sender-authentication requirement. https://developers.google.com/tink/hybrid

[15] Google Tink. *Wire format*. Qualify framing and recipient-library interoperability. https://developers.google.com/tink/wire-format

[16] IRTF/CFRG. *RFC 9180: Hybrid Public Key Encryption*. https://www.rfc-editor.org/rfc/rfc9180.html

[17] IETF. *RFC 8725: JSON Web Token Best Current Practices*. Issuer/audience and indirect-key-lookup security. https://www.rfc-editor.org/rfc/rfc8725.html

[18] IETF. *RFC 9449: OAuth 2.0 Demonstrating Proof of Possession (DPoP)*. https://www.rfc-editor.org/rfc/rfc9449.html

[19] Cerbos. *Public policy schema at the mapped revision*. Interoperability contract reference; independent implementation required. https://github.com/cerbos/cerbos/blob/ae014ba2ea919771216bc666c4f654d18a1c1a52/api/public/cerbos/policy/v1/policy.proto#L35

[20] Go project. *Effective Go*. Naming, interfaces and idiomatic implementation guidance. https://go.dev/doc/effective_go

[21] Go standard library. *net/http: ServeMux*. Path-cleaning behavior must be considered for S3 routing. https://pkg.go.dev/net/http#ServeMux

[22] Jack Christensen and contributors. *pgx: PostgreSQL driver and toolkit for Go*. https://github.com/jackc/pgx

[23] sqlc project. *Compile SQL to type-safe code*. Build-time generator; see repository for MIT license. https://sqlc.dev/ and https://github.com/sqlc-dev/sqlc/blob/main/LICENSE

[24] CoreOS and contributors. *Go OpenID Connect client*. https://github.com/coreos/go-oidc

[25] Go JOSE project. *JOSE standards implementation for Go*. https://github.com/go-jose/go-jose

[26] Santhosh Tekuri and contributors. *JSON Schema validation for Go*. https://github.com/santhosh-tekuri/jsonschema

[27] YAML organization. *Maintained Go YAML implementation*. Select supported version and preserve notices. https://github.com/yaml/go-yaml

[28] Model Context Protocol. *Official Go SDK*. Protocol-version compatibility and package responsibilities. https://github.com/modelcontextprotocol/go-sdk

[29] Model Context Protocol. *Go SDK license*. MIT-to-Apache transition; evaluate the selected pin. https://github.com/modelcontextprotocol/go-sdk/blob/main/LICENSE

[30] Go project. *Security resources*. https://go.dev/doc/security/

[31] Go project. *Vulnerability management and govulncheck*. https://go.dev/doc/security/vuln/

[32] Go project. *Go fuzzing*. https://go.dev/doc/security/fuzz/

[33] Go project. *Data race detector*. Runtime coverage limits and usage. https://go.dev/doc/articles/race_detector

[34] US Copyright Office. *Circular 33: Works Not Protected by Copyright*. Ideas and protected expression; jurisdiction-specific background. https://www.copyright.gov/circs/circ33.pdf

[35] European Union. *Legal protection: Computer programs*. Official summary of Directive 2009/24/EC, including ideas/principles and lawful study. https://eur-lex.europa.eu/legal-content/EN/ALL/?uri=legissum:l26027

[36] Apache Software Foundation. *Apache License 2.0*, especially section 4. https://www.apache.org/licenses/LICENSE-2.0

[37] MinIO community source. Research baseline commit `7aac2a2c5b7c882e68c1ce017d8256be2feea27f`. Reference material only; no source incorporation planned. https://github.com/minio/minio/tree/7aac2a2c5b7c882e68c1ce017d8256be2feea27f

[38] Cerbos OSS PDP source. Completed mapping baseline, commit `ae014ba2ea919771216bc666c4f654d18a1c1a52`. Reference material only; no source incorporation planned. https://github.com/cerbos/cerbos/tree/ae014ba2ea919771216bc666c4f654d18a1c1a52

[39] Go project. *Download manifest*, consulted 30 October 2026; initial compiler baseline Go 1.27.3. https://go.dev/dl/?mode=json

[40] CEL project. *CEL-Go v0.33.1 release*. Module `cel.dev/cel-go`; observed commit `fd8eb2343c6a4294575aef95a9ad03f82b5ba6ca`. https://github.com/cel-expr/cel-go/releases/tag/v0.33.1

[41] CEL project. *License at inspected commit*. Apache 2.0. https://github.com/cel-expr/cel-go/blob/fd8eb2343c6a4294575aef95a9ad03f82b5ba6ca/LICENSE

[42] CEL project. *Program/environment options at inspected commit*. Cost, parser and evaluation controls. https://github.com/cel-expr/cel-go/blob/fd8eb2343c6a4294575aef95a9ad03f82b5ba6ca/cel/options.go

[43] Tink project. *Tink Go v2.9.0 release*. https://github.com/tink-crypto/tink-go/releases/tag/v2.9.0

[44] Tink project. *Tink Go v2.9.0 license*. Apache 2.0. https://github.com/tink-crypto/tink-go/blob/v2.9.0/LICENSE

[45] Tink project. *Streaming AEAD templates, v2.9.0*. https://github.com/tink-crypto/tink-go/blob/v2.9.0/streamingaead/streamingaead_key_templates.go

[46] Google Tink. *AES-GCM-HKDF Streaming format*. https://developers.google.com/tink/streaming-aead/aes_gcm_hkdf_streaming

[47] OpenBao project. *v2.7.2 release*; candidate external service. https://github.com/openbao/openbao/releases/tag/v2.7.2

[48] OpenBao. *Transit Secrets Engine API*, version 2.7.x documentation. https://openbao.org/docs/api/secret/transit/

[49] OpenBao. *Integrated Storage*. Replication, consensus and recovery reference. https://openbao.org/docs/concepts/integrated-storage/

[50] AWS. *Concepts in the AWS Encryption SDK*. Envelope encryption and authenticated context; pattern reference, not a mandatory AWS dependency. https://docs.aws.amazon.com/encryption-sdk/latest/developer-guide/concepts.html

[51] AWS. *AWS SDK for Go v2 license*. Apache 2.0; exact client modules must be pinned at bootstrap. https://github.com/aws/aws-sdk-go-v2/blob/main/LICENSE.txt

[52] AWS. *Go v2 SigV4 signer*. Client signing implementation; not a complete server verifier. https://github.com/aws/aws-sdk-go-v2/blob/main/aws/signer/v4/v4.go

[53] AWS. *Signature calculations for the Authorization header: single-chunk payload*. https://docs.aws.amazon.com/AmazonS3/latest/API/sig-v4-header-based-auth.html

[54] AWS. *PutObject API*. https://docs.aws.amazon.com/AmazonS3/latest/API/API_PutObject.html

[55] AWS. *Conditional writes*. https://docs.aws.amazon.com/AmazonS3/latest/userguide/conditional-writes.html

[56] AWS. *ListObjectsV2 API*. https://docs.aws.amazon.com/AmazonS3/latest/API/API_ListObjectsV2.html

[57] AWS. *CopyObject API*. Future compatibility work. https://docs.aws.amazon.com/AmazonS3/latest/API/API_CopyObject.html

[58] AWS. *CompleteMultipartUpload API*. Future compatibility work. https://docs.aws.amazon.com/AmazonS3/latest/API/API_CompleteMultipartUpload.html

[59] Model Context Protocol. *Authorization*, protocol revision 2026-07-28. https://modelcontextprotocol.io/specification/2026-07-28/basic/authorization

[60] Model Context Protocol. *Tools*, protocol revision 2026-07-28. https://modelcontextprotocol.io/specification/2026-07-28/server/tools

[61] IETF. *RFC 8693: OAuth 2.0 Token Exchange*. Delegation and actor/subject concepts. https://www.rfc-editor.org/rfc/rfc8693.html

[62] IETF. *RFC 9396: OAuth 2.0 Rich Authorization Requests*. Structured authorization details. https://www.rfc-editor.org/rfc/rfc9396.html

[63] PostgreSQL. *Log-Shipping Standby Servers*, PostgreSQL 18 documentation. Synchronous replication and failover background. https://www.postgresql.org/docs/18/warm-standby.html

[64] PostgreSQL. *Write-ahead log settings*. Durable synchronous-commit behavior. https://www.postgresql.org/docs/18/runtime-config-wal.html

[65] Patroni. *Replication modes*. Strict synchronous mode, failover and commit caveats. https://patroni.readthedocs.io/en/latest/replication_modes.html

[66] OpenBao. *Integrated storage internals*. Quorum and deployment-size guidance. https://openbao.org/docs/internals/integrated-storage/

[67] V. C. Hu. *Overview and Considerations of Access Control Based on Attribute Encryption*. NIST IR 8450-upd1, 2023. https://nvlpubs.nist.gov/nistpubs/ir/2023/NIST.IR.8450-upd1.pdf

[68] J. Bethencourt, A. Sahai, B. Waters. *Ciphertext-Policy Attribute-Based Encryption*. IEEE Symposium on Security and Privacy, 2007. https://www.cs.ucla.edu/~sahai/work/web/2007%20Publications/SSP2007.pdf